

Student Name: Juntang Wang

Student ID: jw853

Part I

Problem 1. backpropagation

Solution:

The forward pass can be described as:

- Linear layer: $z_1 = w_1x$, $z_2 = w_2x$
- Activation function: $h_1 = \sigma(z_1)$, $h_2 = \sigma(z_2)$
- Output layer: $y = h_1 + h_2$

As we want $\partial y / \partial x$, we can use the chain rule to get:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial x} + \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial x}$$

For each of the terms, we have:

- $\frac{\partial y}{\partial h_1} = 1$
- $\frac{\partial y}{\partial h_2} = 1$
- $\frac{\partial h_1}{\partial z_1} = \sigma'(z_1)$
- $\frac{\partial h_2}{\partial z_2} = \sigma'(z_2)$
- $\frac{\partial z_1}{\partial x} = w_1$
- $\frac{\partial z_2}{\partial x} = w_2$

Thus we have:

$$\frac{\partial y}{\partial x} = \sigma'(z_1)w_1 + \sigma'(z_2)w_2$$

Problem 2. batch normalization

In "Lecture 2 Notebook 1", we defined a neural network with the following architecture: [input] -> Linear(28x28, 512) -> ReLU -> Linear(512, 512) -> ReLU* -> Linear(512, 10) -> ReLU -> [output]

In this exercise, we try to reproduce Page 3 of Lecture 2's slides. First, use the above network to train for 50 epochs. Plot the histogram of activation values of the second linear layer (the values of "*" above), versus epochs.

Then, add batch normalization as in "Lecture 2 Notebook 1". Plot the histogram of the activation after the second batch normalization, versus epochs.

Compare the two plots.

Solution:

below we define the nn using the provided code

```
In [12]: import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor
import numpy as np
import matplotlib.pyplot as plt
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class NeuralNetwork(nn.Module):
    def __init__(self, use_batchnorm=0):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.use_batchnorm = use_batchnorm
        layers = [
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(), # this is the * in the question
            nn.Linear(512, 10),
            nn.ReLU()
        ]
        if use_batchnorm:
            layers.insert(1, nn.BatchNorm1d(512))
            layers.insert(3, nn.BatchNorm1d(512))
        self.linear_relu_stack = nn.Sequential(*layers)

    def forward(self, x):
        x = self.flatten(x)
        return self.linear_relu_stack(x)

    def get_star_values(self, x):
        x = self.flatten(x)
        num_layers = 4 + 2*self.use_batchnorm # +1 for batchnorm
        x = self.linear_relu_stack[:num_layers](x)
        return x

training_data = datasets.FashionMNIST(root="data", train=True, download=True, transform=T
test_data = datasets.FashionMNIST(root="data", train=False, download=True, transform=T
train_dataloader = DataLoader(training_data, batch_size=64)
test_dataloader = DataLoader(test_data, batch_size=64)

def train_loop(dataloader, model, loss_fn, optimizer):
    for X, y in dataloader:
        X, y = X.to(device), y.to(device)
        pred = model(X)
        loss = loss_fn(pred, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

def star_values(dataloader, model):
    activations = []
```

```

with torch.no_grad():
    for X, y in dataloader:
        X = X.to(device)
        batch_activations = model.get_star_values(X)
        activations.append(batch_activations.cpu())
    return torch.cat(activations, dim=0)

def test_loop(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0

    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f'Test Error: Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f}')

# Setup
fig = plt.figure(figsize=(15, 10))
axes_3d = [fig.add_subplot(2, 2, i, projection='3d') for i in [1, 2]]
axes_2d = [fig.add_subplot(2, 2, i) for i in [3, 4]]
lr = 1e-3
epochs = 50

for i, use_batchnorm in enumerate([False, True]):
    model = NeuralNetwork(use_batchnorm=use_batchnorm).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, betas=(0.9, 0.99))
    loss_fn = nn.CrossEntropyLoss()

    all_activations = []
    for t in tqdm(range(epochs)):
        train_loop(train_dataloader, model, loss_fn, optimizer)
        epoch_activations = star_values(test_dataloader, model)
        all_activations.append(epoch_activations)

    test_loop(test_dataloader, model, loss_fn)

# Plot 3D histogram
ax_3d = axes_3d[i]
heatmap_data = np.zeros((50, epochs))
bin_edges = None
for epoch in range(epochs):
    activations = all_activations[epoch].flatten().numpy()
    # Filter out zeros for better visualization
    activations = activations[np.abs(activations) > 1e-6]
    hist, bins = np.histogram(activations, bins=50)
    bin_centers = (bins[:-1] + bins[1:]) / 2
    ax_3d.bar(bin_centers, hist, zs=epoch, zdir='x', alpha=0.7)
    heatmap_data[:, epoch] = hist
    if bin_edges is None:
        bin_edges = bins
ax_3d.set_xlabel('Epoch')
ax_3d.set_ylabel('Activation Value')
ax_3d.set_zlabel('Frequency')
ax_3d.set_title(f'{"With" if use_batchnorm else "Without"} Batch Normalization -')

# Plot 2D heatmap (top-down view)

```

```

ax_2d = axes_2d[i]
im = ax_2d.imshow(heatmap_data, aspect='auto', origin='lower',
                  extent=[0, epochs-1, bin_edges[0], bin_edges[-1]],
                  cmap='viridis')
ax_2d.set_xlabel('Epoch')
ax_2d.set_ylabel('Activation Value')
ax_2d.set_title(f'{"With" if use_batchnorm else "Without"} Batch Normalization -')
plt.colorbar(im, ax=ax_2d, label='Frequency')

plt.tight_layout()
plt.show()

print("Done!")

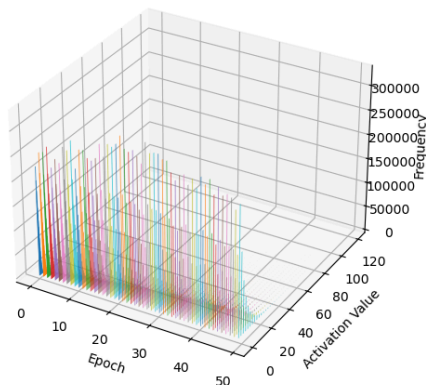
```

```

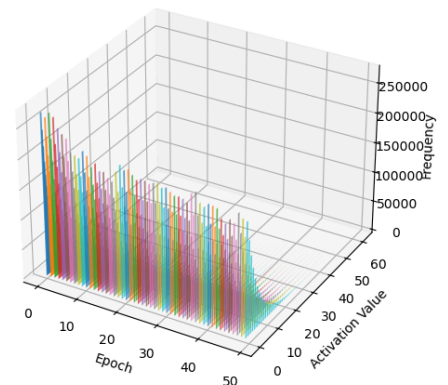
0%|          | 0/50 [00:00<?, ?it/s]
Test Error: Accuracy: 79.5%, Avg loss: 1.052326
0%|          | 0/50 [00:00<?, ?it/s]
Test Error: Accuracy: 88.5%, Avg loss: 1.055724

```

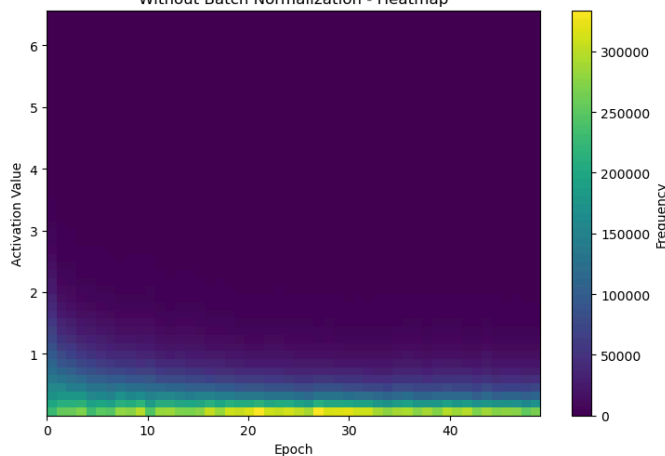
Without Batch Normalization - 3D View



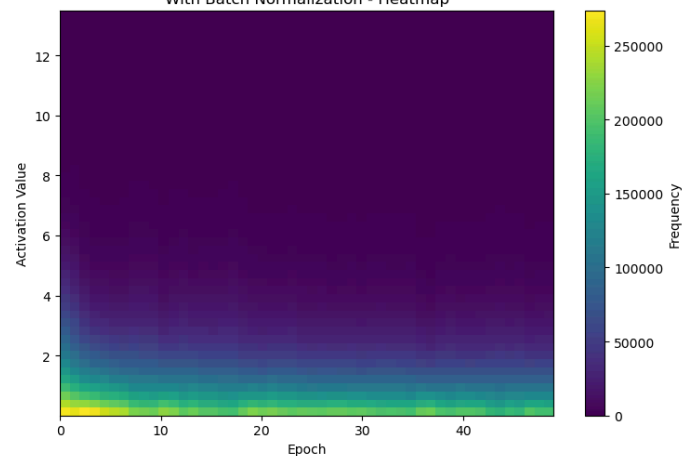
With Batch Normalization - 3D View



Without Batch Normalization - Heatmap



With Batch Normalization - Heatmap



Done!

Observation:

- The activation values are more concentrated (skewed) around 0 without batch normalization.
- The activation values are more spread out with batch normalization.
- The activation values are more concentrated (skewed) around 0 without batch normalization.
- The activation values are more spread out with batch normalization.

Intuitively, batch normalization helps to normalize, so that the distribution of activation values is more apt to be like uniform.

Problem 3. universal approximation

Let $f : [-1, 1]^n \rightarrow [-1, 1]$ be a Lipschitz function (that is, $|f(x) - f(y)| \leq L\|x - y\|$ for some constant L). Prove: given any $\epsilon > 0$, there is a neural network $N : [-1, 1]^n \rightarrow [-1, 1]$ with a sigmoid activation function such that for every $x \in [-1, 1]^n$, it holds that

$$|f(x) - N(x)| \leq \epsilon$$

1. Lipschitz $\Rightarrow$ Uniform Continuity

Since f is L -Lipschitz on the compact domain $[-1, 1]^n$, it is uniformly continuous. Thus, for any $\epsilon > 0$, there exists $\delta > 0$ such that

$$\|x - y\| \leq \delta \implies |f(x) - f(y)| \leq \frac{\epsilon}{2}.$$

2. Construct a Piecewise-Constant Approximation

1. Partition

Partition $[-1, 1]^n$ into finitely many small cubes $\{C_k\}_{k=1}^M$, each of diameter $\leq \delta$. (Hence, any two points in the same cube are at most δ apart.)

2. Define g

For each cube C_k , choose a representative point $x_k \in C_k$. Define

$$g(x) := \sum_{k=1}^M f(x_k) \mathbf{1}_{C_k}(x),$$

where $\mathbf{1}_{C_k}(x)$ is the indicator of C_k .

3. Approximation Error

If $x \in C_k$, then $\|x - x_k\| \leq \delta$. By Lipschitz continuity,

$$|f(x) - f(x_k)| \leq L\|x - x_k\| \leq L\delta \leq \frac{\epsilon}{2},$$

provided $\delta \leq \epsilon/(2L)$. Thus,

$$\max_{x \in [-1, 1]^n} |f(x) - g(x)| \leq \frac{\epsilon}{2}.$$

3. Approximate g by a Sigmoid Network

We need to approximate the step-function-like terms $\mathbf{1}_{C_k}(x)$ using sigmoids, then sum them appropriately.

1. Approximate a 1D Step

In one dimension, consider $\mathbf{1}_{[a, b]}(x)$. A sigmoid $\sigma(t)$ with large slope can approximate the Heaviside step:

$$\sigma(A(x - a)) \approx \begin{cases} 1, & x > a \\ 0, & x < a \end{cases} \quad \text{for large } A,$$

and similarly, $\sigma(A(x - b))$ approximates $\mathbf{1}_{x > b}$. By difference, one can approximate $\mathbf{1}_{[a, b]}(x) \approx \sigma(A(x - a)) - \sigma(A(x - b))$.

2. Indicator of an n -D Cube

Each n -dimensional cube $C_k = \prod_{j=1}^n [a_j^{(k)}, b_j^{(k)}]$ can be written as

$$C_k = \{x : a_j^{(k)} \leq x_j \leq b_j^{(k)} \text{ for all } j\}.$$

Hence

$$\mathbf{1}_{C_k}(x) = \prod_{j=1}^n \mathbf{1}_{[a_j^{(k)}, b_j^{(k)}]}(x_j).$$

A feedforward network can approximate each 1D indicator and then approximate their product (logical AND) by using additional sigmoid layers that emulate multiplication (e.g., via standard constructions in universal approximation theory).

3. Summation

Once we have $\tilde{\mathbf{1}}_{C_k}(x) \approx \mathbf{1}_{C_k}(x)$, define

$$N(x) := \sum_{k=1}^M f(x_k) \tilde{\mathbf{1}}_{C_k}(x).$$

By adjusting the network parameters (increasing widths, adjusting slopes), we can make $\tilde{\mathbf{1}}_{C_k}$ arbitrarily close to $\mathbf{1}_{C_k}$. Thus,

$$\max_{x \in [-1, 1]^n} |g(x) - N(x)| \leq \frac{\varepsilon}{2}.$$

4. Combine the Errors

Finally,

$$|f(x) - N(x)| \leq |f(x) - g(x)| + |g(x) - N(x)| \leq \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

Hence,

$$\max_{x \in [-1, 1]^n} |f(x) - N(x)| \leq \varepsilon.$$

Conclusion

Every Lipschitz function on the compact domain $[-1, 1]^n$ can be approximated uniformly by a sufficiently large **sigmoid** neural network. This completes the proof of the universal approximation property for f .

Problem 4. regularization

Suppose the objective function in training is $J(\theta; X, y)$, where θ represents the parameters, X is the training data input, and y is target data output. One commonly used regularization is L^2 regularization, where the objective function is

$$\tilde{J}(w; X, y) = J(w; X, y) + \frac{\alpha}{2} w^T w$$

whose gradient w.r.t. w is

$$\nabla \tilde{J}(w; X, y) = \nabla_w J(w; X, y) + \alpha w$$

1. Write the iteration step in the gradient descent method.
2. Assume the objective function is quadratic and $w^* = \arg \min_w J(w)$, then

$$J(w) = J(w^*) + \frac{1}{2}(w - w^*)^T H (w - w^*)$$

Show that the minimizer of $\tilde{J}$ is given by

$$\tilde{w} = (H + \alpha I)^{-1} H w^*$$

3. Show that $\tilde{w}$ is a scaled version of w^* in the sense that along the direction of the i -th eigenvector of H , w^* is scaled by a factor of

$$\frac{\lambda_i}{\lambda_i + \alpha}$$

1. Gradient Descent Iteration

The **gradient descent** update for $\tilde{J}$ at iteration k is

$$w^{(k+1)} = w^{(k)} - \eta \left(\nabla_w J(w^{(k)}) + \alpha w^{(k)} \right),$$

where η is the learning rate.

2. Minimizer When J is Quadratic

Assume

$$J(w) = J(w^*) + \frac{1}{2} (w - w^*)^\top H (w - w^*),$$

with $w^* = \arg \min_w J(w)$. Then

$$\tilde{J}(w) = J(w) + \frac{\alpha}{2} \|w\|^2 = J(w^*) + \frac{1}{2} (w - w^*)^\top H (w - w^*) + \frac{\alpha}{2} \|w\|^2.$$

To find its minimizer $\tilde{w}$, set the gradient to zero:

$$\nabla \tilde{J}(w) = H (w - w^*) + \alpha w = 0.$$

Rearranging,

$$Hw - Hw^* + \alpha w = 0 \implies (H + \alpha I)w = Hw^*.$$

Thus,

$$\boxed{\tilde{w} = (H + \alpha I)^{-1} H w^*}.$$

3. $\tilde{w}$ is a Scaled Version of w^* Along Each Eigenvector

Let $H = Q \Lambda Q^\top$ be the eigendecomposition of H , with $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$. In that basis,

$$(H + \alpha I)^{-1} H = (Q (\Lambda + \alpha I) Q^\top)^{-1} \cdot Q \Lambda Q^\top = Q \operatorname{diag}\left(\frac{\lambda_i}{\lambda_i + \alpha}\right) Q^\top.$$

Hence on the i -th eigenvector q_i , the vector $\tilde{w}$ is exactly

$$\tilde{w}_i = \frac{\lambda_i}{\lambda_i + \alpha} w_i^*.$$

In other words, **each eigen-component** of w^* is **scaled** by $\frac{\lambda_i}{\lambda_i + \alpha}$.

This completes the proof.

Part II

1 Convolution

1.1 (6 points)

Consider a rectangular signal which has value 1 for some finite duration $L > 1$, and value 0 everywhere else. First, compute by hand the convolution of two such rectangular signals. Then, write a Python function `convolve(x, h)` which computes the convolution of two signals of the same length (though not necessarily rectangular). Finally, graph the output of your function when passing two rectangular signals as input. Does it match the expectations of your original computation? Please include a copy of your code and output with your submission.

1.1 Solution

Let's first compute the convolution of two rectangular signals by hand.

Let $x(n)$ and $h(n)$ be rectangular signals of length L , where:

$$x(n) = \begin{cases} 1, & a \leq n < a + L \\ 0, & \text{otherwise} \end{cases}$$

$$h(n) = \begin{cases} 1, & b \leq n < b + L \\ 0, & \text{otherwise} \end{cases}$$

The convolution $y(n) = (x * h)(n) = \sum_{k=-\infty}^{\infty} x(k)h(n - k)$

For our rectangular signals, we can simplify this to:

$$y(n) = \sum_{k=a}^{a+L-1} x(k)h(n - k) = \sum_{k=a}^{a+L-1} h(n - k) \text{ for } a \leq k < a + L$$

For different values of n :

- When $n < a + b$ or $n \geq a + b + 2L$: $y(n) = 0$ (no overlap)
- When $a + b \leq n < a + b + L$: $y(n) = n - (a + b) + 1$ (partial overlap, increasing)
- When $a + b + L \leq n < a + b + 2L$: $y(n) = a + b + 2L - n - 1$ (partial overlap, decreasing)

This gives us a triangular signal with peak value L at $n = a + b + L - 1$.

Now let's implement the convolution function and verify our result:

```
In [15]: import numpy as np
import matplotlib.pyplot as plt

def convolve(x, h):
    """Compute the discrete convolution of two signals x and h."""
    x = np.asarray(x)
    h = np.asarray(h)
    y_length = len(x) + len(h) - 1
    y = np.zeros(y_length)

    for n in range(y_length):
        for k in range(len(x)):
            if 0 <= n-k < len(h):
                y[n] += x[k] * h[n-k]

    return y

# Verify with rectangular signals as described in the solution
L = 5
x_rect = np.ones(L)
h_rect = np.ones(L)
y_rect = convolve(x_rect, h_rect)

# Create continuous-like signals with more samples
L_continuous = 100
x_continuous = np.ones(L_continuous)
h_continuous = np.ones(L_continuous)
y_continuous = np.convolve(x_continuous, h_continuous, mode='full')

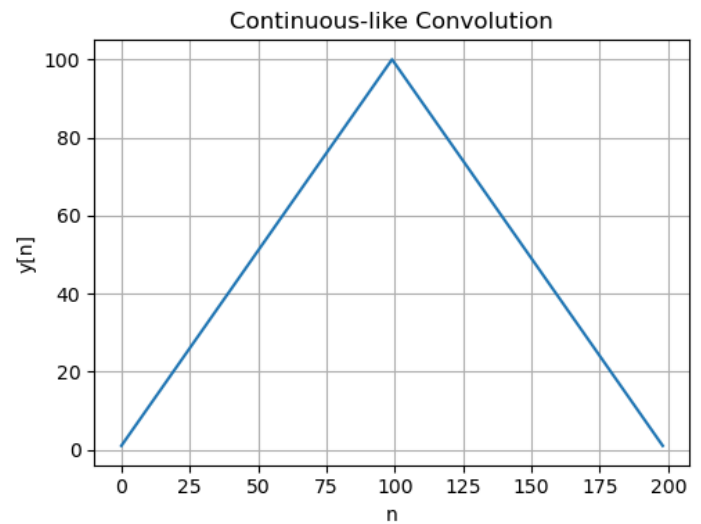
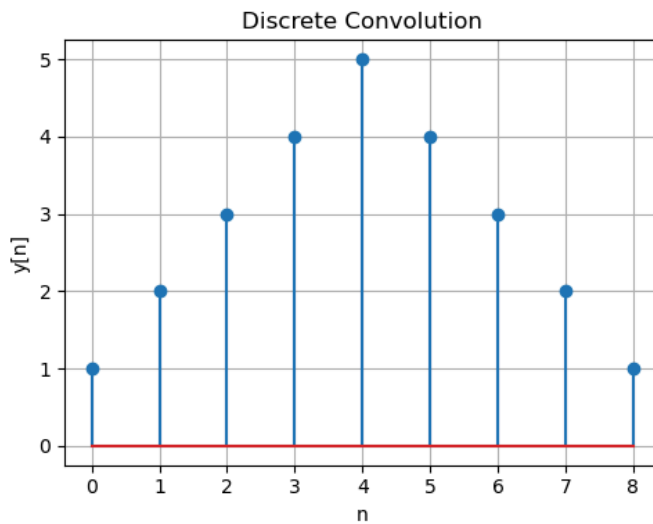
# Plot both discrete and continuous versions in two panels
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

# Discrete version
ax1.stem(range(len(y_rect)), y_rect)
ax1.set_title('Discrete Convolution')
ax1.set_xlabel('n')
ax1.set_ylabel('y[n]')
ax1.grid(True)

# Continuous version
ax2.plot(range(len(y_continuous)), y_continuous)
ax2.set_title('Continuous-like Convolution')
ax2.set_xlabel('n')
ax2.set_ylabel('y[n]')
ax2.grid(True)

plt.tight_layout()
plt.show()

# Verify the result matches our theoretical analysis
expected_values = [i+1 if i < L else 2*L-i-1 for i in range(2*L-1)]
print(f"Computed values: {y_rect}")
print(f"Expected values: {expected_values}")
print(f"Match: {np.allclose(y_rect, expected_values)}")
```



Computed values: [1. 2. 3. 4. 5. 4. 3. 2. 1.]

Expected values: [1, 2, 3, 4, 5, 4, 3, 2, 1]

Match: True

Observation

The discrete convolution matches the theoretical solution, showing a triangular shape with peak value L at $n = L-1$. The continuous version using numpy's convolve function also matches the theoretical solution, showing a smooth, continuous triangular shape.

1.2 (2 points)

If $y(n) = x(n) * h(n)$, show that $\sum_{n=-\infty}^{\infty} y(n) = (\sum_{n=-\infty}^{\infty} x(n))(\sum_{n=-\infty}^{\infty} h(n))$.

1.2 Solution

To show that $\sum_{n=-\infty}^{\infty} y(n) = (\sum_{n=-\infty}^{\infty} x(n))(\sum_{n=-\infty}^{\infty} h(n))$ where $y(n) = x(n) * h(n)$:

Starting with the definition of discrete convolution: $y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$

Taking the sum of $y(n)$ over all n : $\sum_{n=-\infty}^{\infty} y(n) = \sum_{n=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} x(k)h(n-k)$

Rearranging the order of summation: $\sum_{n=-\infty}^{\infty} y(n) = \sum_{k=-\infty}^{\infty} x(k) \sum_{n=-\infty}^{\infty} h(n-k)$

Let's substitute $m = n - k$, then $n = m + k$. When n ranges from $-\infty$ to ∞ , m also ranges from $-\infty$ to ∞ : $\sum_{n=-\infty}^{\infty} y(n) = \sum_{k=-\infty}^{\infty} x(k) \sum_{m=-\infty}^{\infty} h(m)$

Since $\sum_{m=-\infty}^{\infty} h(m)$ is independent of k , we can take it out of the summation:

$$\sum_{n=-\infty}^{\infty} y(n) = \sum_{k=-\infty}^{\infty} x(k) \cdot \sum_{m=-\infty}^{\infty} h(m)$$

This simplifies to: $\sum_{n=-\infty}^{\infty} y(n) = (\sum_{k=-\infty}^{\infty} x(k))(\sum_{m=-\infty}^{\infty} h(m))$

Which is equivalent to: $\sum_{n=-\infty}^{\infty} y(n) = (\sum_{n=-\infty}^{\infty} x(n))(\sum_{n=-\infty}^{\infty} h(n))$

Therefore, the sum of the convolution equals the product of the sums of the individual signals.

1.3 (4 points)

The equation for the discrete convolution of two signals is:

$$(x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

In the case of finite length signals, we assume both sides of the signal to be zero-padded.

Compute the convolution of the following signals. As a hint, you should expect your answer to be a vector written similarly to $x(n)$ and $h(n)$, with a length one less than the sum of the length of both inputs.

1. $x(n) = [1, 2, 4]$, $h(n) = [1, 1, 1, 1, 1]$
2. $x(n) = [1, 2, -1]$, $h(n) = x(n)$

1.3 Solution

1. For $x(n) = [1, 2, 4]$ and $h(n) = [1, 1, 1, 1, 1]$:

Using the convolution formula $(x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$:

The result will have length $3+5-1 = 7$.

Computing each term:

- $(x * h)[0] = x[0]h[0] = 1 \cdot 1 = 1$
- $(x * h)[1] = x[0]h[1] + x[1]h[0] = 1 \cdot 1 + 2 \cdot 1 = 3$
- $(x * h)[2] = x[0]h[2] + x[1]h[1] + x[2]h[0] = 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 1 = 7$
- $(x * h)[3] = x[0]h[3] + x[1]h[2] + x[2]h[1] = 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 1 = 7$
- $(x * h)[4] = x[0]h[4] + x[1]h[3] + x[2]h[2] = 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 1 = 7$
- $(x * h)[5] = x[1]h[4] + x[2]h[3] = 2 \cdot 1 + 4 \cdot 1 = 6$
- $(x * h)[6] = x[2]h[4] = 4 \cdot 1 = 4$

Therefore, $(x * h)[n] = [1, 3, 7, 7, 7, 6, 4]$

2. For $x(n) = [1, 2, -1]$ and $h(n) = x(n)$:

The result will have length $3+3-1 = 5$.

Computing each term:

- $(x * h)[0] = x[0]h[0] = 1 \cdot 1 = 1$
- $(x * h)[1] = x[0]h[1] + x[1]h[0] = 1 \cdot 2 + 2 \cdot 1 = 4$
- $(x * h)[2] = x[0]h[2] + x[1]h[1] + x[2]h[0] = 1 \cdot (-1) + 2 \cdot 2 + (-1) \cdot 1 = 2$
- $(x * h)[3] = x[1]h[2] + x[2]h[1] = 2 \cdot (-1) + (-1) \cdot 2 = -4$
- $(x * h)[4] = x[2]h[2] = (-1) \cdot (-1) = 1$

Therefore, $(x * h)[n] = [1, 4, 2, -4, 1]$

```
In [18]: # compute and verify by code
# Problem 1
x1 = np.array([1, 2, 4])
h1 = np.array([1, 1, 1, 1, 1])
conv1 = np.convolve(x1, h1, mode='full')
print("1. x(n) = [1,2,4], h(n) = [1,1,1,1,1]")
print(f"    Convolution result: {conv1}")

# Problem 2
```

```

x2 = np.array([1, 2, -1])
h2 = x2 #  $h(n) = x(n)$ 
conv2 = np.convolve(x2, h2, mode='full')
print("\n2.  $x(n) = [1, 2, -1]$ ,  $h(n) = x(n)$ ")
print(f"    Convolution result: {conv2}")

# Verify our manual calculations
assert np.array_equal(conv1, np.array([1, 3, 7, 7, 7, 6, 4]))
assert np.array_equal(conv2, np.array([1, 4, 2, -4, 1]))

```

1. $x(n) = [1, 2, 4]$, $h(n) = [1, 1, 1, 1]$
Convolution result: [1 3 7 7 7 6 4]
2. $x(n) = [1, 2, -1]$, $h(n) = x(n)$
Convolution result: [1 4 2 -4 1]

1.4 (4 points)

Compute and plot the convolutions $x(n) * h(n)$ and $h(n) * x(n)$ for the pairs of signals shown in Figure 1.

Figure 1 description: Panel (a):

- $x(n) = [1, 1, 1, 1]$ (unit pulse of length 4 at $n = 0, 1, 2, 3$)
- $h(n) = [6, 5, 4, 3, 2, 1, 0]$ (linear ramp from 6 to 0 at $n = 0, 1, 2, 3, 4, 5, 6$)

Panel (b):

- $x(n) = [1, 1, 1, 1]$ (unit pulse of length 4 at $n = 0, 1, 2, 3$)
- $h(n) = [6, 5, 4, 3, 2, 1, 0]$ (linear ramp from 6 to 0 at $n = -3, -2, -1, 0, 1, 2, 3$)

1.4 Solution

```

In [21]: import matplotlib.pyplot as plt
import numpy as np

# Define signals
x = np.array([1, 1, 1, 1]) # unit pulse of length 4
h_a = np.array([6, 5, 4, 3, 2, 1, 0]) # linear ramp for panel (a)

# Compute convolutions
x_conv_h = np.convolve(x, h_a, mode='full') #  $x(n) * h(n)$ 
h_conv_x = np.convolve(h_a, x, mode='full') #  $h(n) * x(n)$ 

# Create figure with 4 subplots (2x2)
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

# Panel (a):  $h(n)$  starts at  $n=0$ 
# Top row:  $x(n) * h(n)$ 
axs[0, 0].stem(range(len(x_conv_h)), x_conv_h)
axs[0, 0].set_title('Panel (a):  $x(n) * h(n)$ ')
axs[0, 0].set_xlabel('n')
axs[0, 0].set_ylabel('Amplitude')
axs[0, 0].grid(True)

# Top row:  $h(n) * x(n)$ 
axs[0, 1].stem(range(len(h_conv_x)), h_conv_x)
axs[0, 1].set_title('Panel (a):  $h(n) * x(n)$ ')
axs[0, 1].set_xlabel('n')
axs[0, 1].grid(True)

```

```

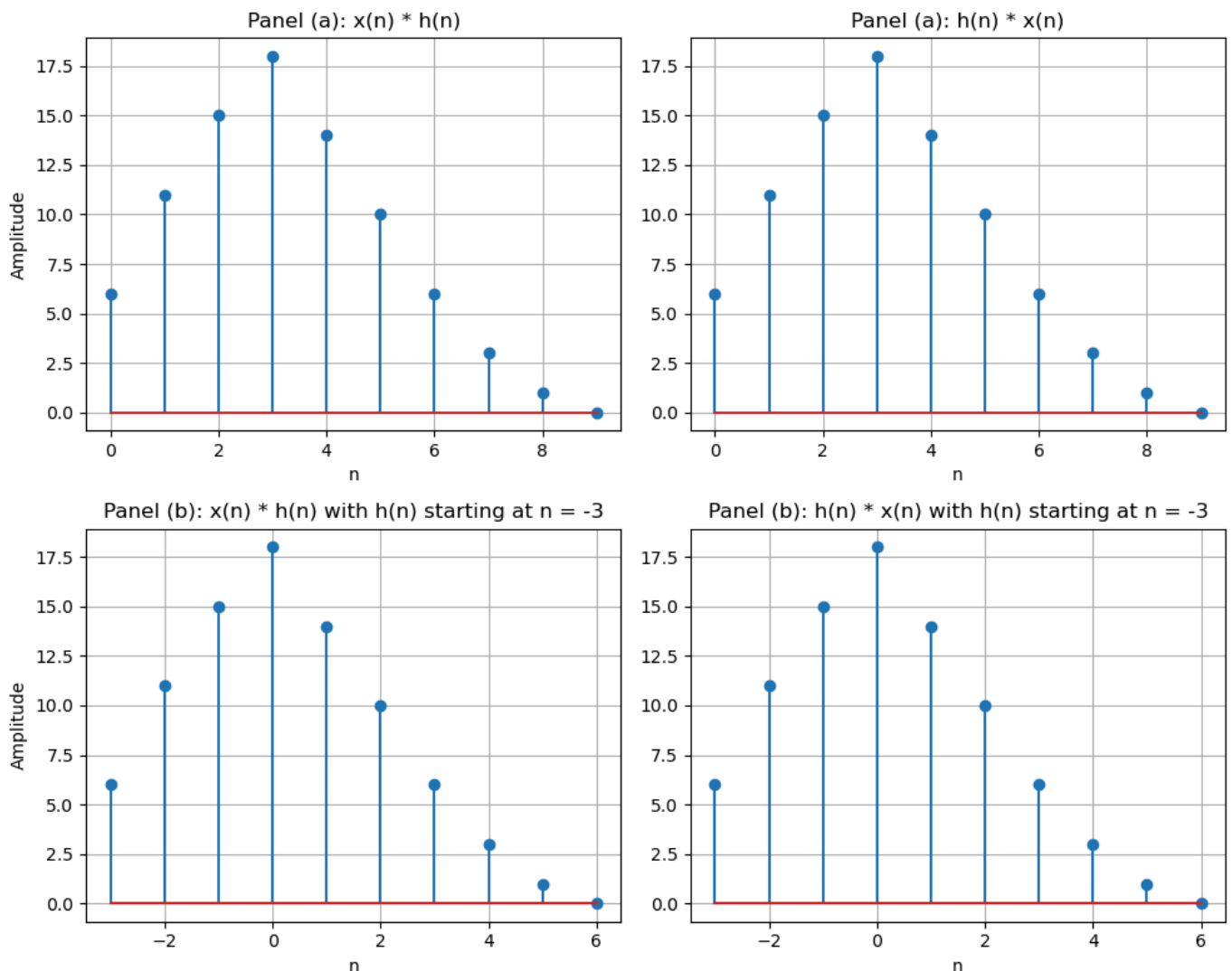
# Panel (b): h(n) starts at n=-3
# Bottom row: x(n) * h(n)
n_indices_b = np.arange(-3, len(x_conv_h)-3)
axs[1, 0].stem(n_indices_b, x_conv_h)
axs[1, 0].set_title('Panel (b): x(n) * h(n) with h(n) starting at n = -3')
axs[1, 0].set_xlabel('n')
axs[1, 0].set_ylabel('Amplitude')
axs[1, 0].grid(True)

# Bottom row: h(n) * x(n)
axs[1, 1].stem(n_indices_b, h_conv_x)
axs[1, 1].set_title('Panel (b): h(n) * x(n) with h(n) starting at n = -3')
axs[1, 1].set_xlabel('n')
axs[1, 1].grid(True)

plt.tight_layout()
plt.show()

# Verify the commutative property
print(f"Commutative property: x(n) * h(n) = h(n) * x(n)? {np.array_equal(x_conv_h, h_

```



Commutative property: $x(n) * h(n) = h(n) * x(n)$? True

2 Backpropagation

In binary classification, we seek to associate an input x_n with its appropriate target value: $t_n = 0$ for class C_0 and $t_n = 1$ for class C_1 . The cross-entropy error function is of the form:

$$E(w) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

Assuming a neural network with output units which use the logistic sigmoid activation function:

$$y_n = \sigma(a_n) = \frac{1}{1 + e^{-a_n}}$$

show the derivative of the above error function satisfies:

$$\frac{\partial E}{\partial a_k} = y_k - t_k$$

Hint: show that $\frac{d\sigma}{da} = \sigma(1 - \sigma)$. Use this relation to simplify the expressions for the derivatives of $\ln(y)$ and $\ln(1 - y)$.

2 Solution

First, let's show that $\frac{d\sigma}{da} = \sigma(1 - \sigma)$:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

Taking the derivative: $\frac{d\sigma}{da} = \frac{e^{-a}}{(1 + e^{-a})^2} = \frac{1}{1 + e^{-a}} \cdot \frac{e^{-a}}{1 + e^{-a}} = \sigma(a) \cdot (1 - \sigma(a))$

Now, for $\frac{\partial E}{\partial a_k}$:

$$E(w) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

For a specific output unit k : $\frac{\partial E}{\partial a_k} = - \left\{ t_k \frac{1}{y_k} \frac{\partial y_k}{\partial a_k} + (1 - t_k) \frac{1}{1 - y_k} \frac{\partial(1 - y_k)}{\partial a_k} \right\}$

Since $\frac{\partial y_k}{\partial a_k} = y_k(1 - y_k)$ and $\frac{\partial(1 - y_k)}{\partial a_k} = -y_k(1 - y_k)$:

$$\frac{\partial E}{\partial a_k} = - \{ t_k(1 - y_k) - (1 - t_k)y_k \} = - \{ t_k - t_k y_k - y_k + t_k y_k \} = - \{ t_k - y_k \} = y_k - t_k$$

Therefore, $\frac{\partial E}{\partial a_k} = y_k - t_k$.

3 Jensen's Inequality

Jensen's inequality states that

$$f\left(\sum_{i=1}^M \lambda_i x_i\right) \leq \sum_{i=1}^M \lambda_i f(x_i)$$

where $\lambda_i > 0$ and $\sum_i \lambda_i = 1$, for any set of points $\{x_i\}$.

Consider an M -state discrete random variable x , with entropy

$$H(x) = - \sum_{i=1}^M p(x_i) \ln p(x_i)$$

Show that the entropy of its distribution $p(x)$ satisfies:

$$H(x) \leq \ln M$$

3 Solution

To prove that $H(x) \leq \ln M$, I'll use Jensen's inequality with the convex function $f(x) = -\ln(x)$.

First, note that $f(x) = -\ln(x)$ is indeed convex for $x > 0$ since its second derivative $f''(x) = \frac{1}{x^2} > 0$.

Let's define $\lambda_i = p(x_i)$ for $i = 1, 2, \dots, M$. Since $p(x_i)$ is a probability distribution, we know that $p(x_i) \geq 0$ and $\sum_{i=1}^M p(x_i) = 1$, satisfying the conditions for Jensen's inequality.

Applying Jensen's inequality with $f(x) = -\ln(x)$:

$$f\left(\sum_{i=1}^M \lambda_i x_i\right) \leq \sum_{i=1}^M \lambda_i f(x_i)$$

Let's set $x_i = \frac{1}{p(x_i)}$ for each i . Then:

$$-\ln\left(\sum_{i=1}^M p(x_i) \cdot \frac{1}{p(x_i)}\right) \leq \sum_{i=1}^M p(x_i) \cdot \left(-\ln\left(\frac{1}{p(x_i)}\right)\right)$$

Simplifying the left side:

$$-\ln\left(\sum_{i=1}^M 1\right) = -\ln(M)$$

Simplifying the right side:

$$\sum_{i=1}^M p(x_i) \cdot \left(-\ln\left(\frac{1}{p(x_i)}\right)\right) = \sum_{i=1}^M p(x_i) \cdot \ln(p(x_i)) = -H(x)$$

Therefore:

$$-\ln(M) \leq -H(x)$$

Multiplying both sides by -1 (which reverses the inequality):

$$\ln(M) \geq H(x)$$

Thus, we have proven that $H(x) \leq \ln M$.

Note: The maximum entropy occurs when all states are equally likely, i.e., when $p(x_i) = \frac{1}{M}$ for all i , in which case $H(x) = \ln M$.

4 Infomation Theory

4.1 Entropy (4 points)

1. For a biased coin with probability $P(head) = p$ and $P(tail) = 1 - p$, show that maximal entropy is achieved when the coin is fair.

Hint: Write the expression of binary entropy as a function of p and find its maximum.

2. Plot $H(p)$ as a function of p , where p takes values from 0 to 1.
-

4.1 Solution

1. The entropy of a binary random variable is defined as:

$$H(p) = -p \log(p) - (1 - p) \log(1 - p)$$

To find the maximum entropy, we take the derivative with respect to p and set it to zero:

$$\frac{dH(p)}{dp} = -\log(p) - 1 + \log(1 - p) + 1 = \log\left(\frac{1-p}{p}\right) = 0$$

This gives us $\frac{1-p}{p} = 1$, which means $1 - p = p$, so $p = 0.5$

To verify this is a maximum (not a minimum), we can check the second derivative:

$$\frac{d^2H(p)}{dp^2} = -\frac{1}{p} - \frac{1}{1-p} < 0 \text{ for } 0 < p < 1$$

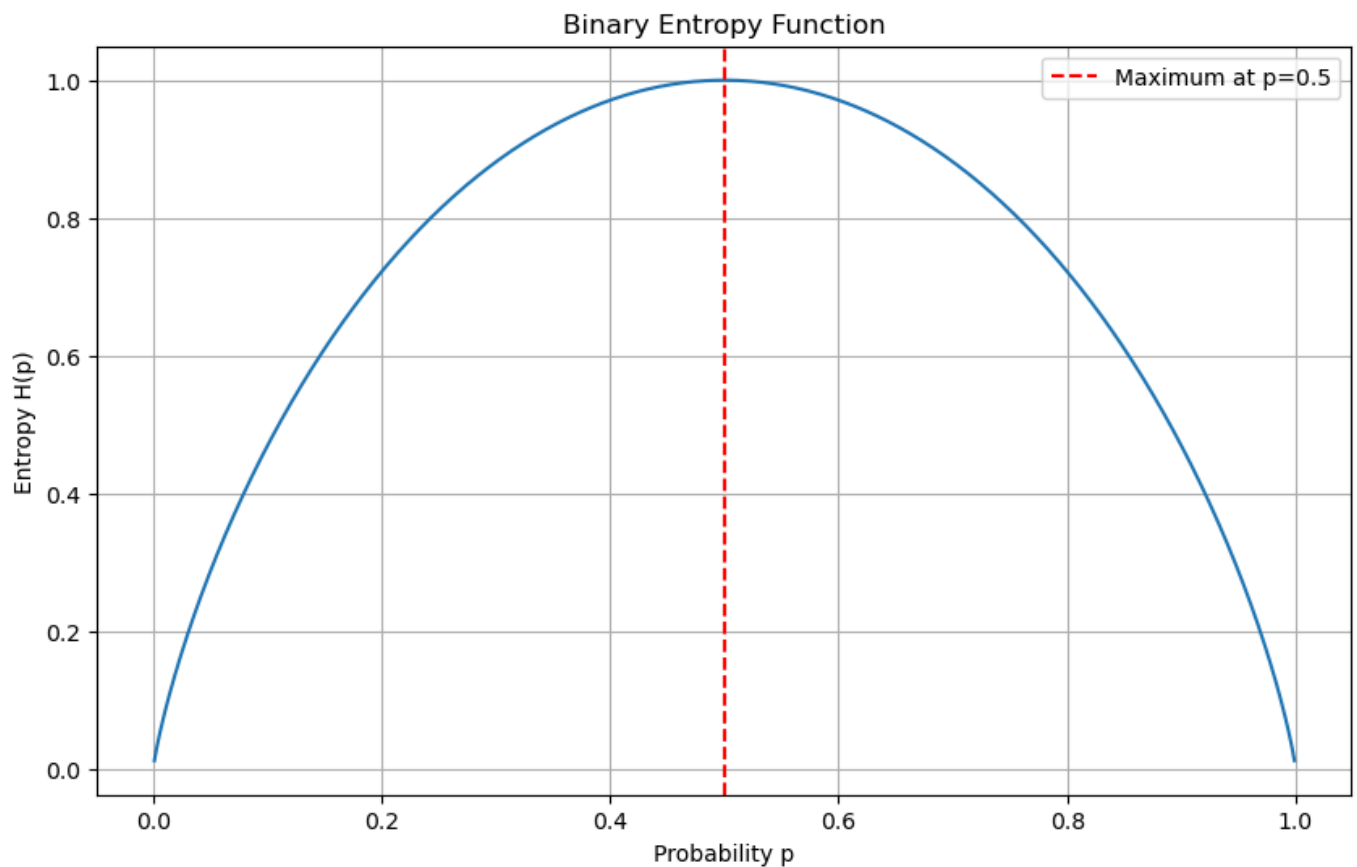
Therefore, the entropy is maximized when $p = 0.5$, i.e., when the coin is fair.

2. Here's the plot of $H(p)$ as a function of p :

```
In [22]: # Define the entropy function
def entropy(p):
    if p == 0 or p == 1:
        return 0
    return -p * np.log2(p) - (1-p) * np.log2(1-p)

# Create an array of p values from 0 to 1
p_values = np.linspace(0.001, 0.999, 1000)
entropy_values = [entropy(p) for p in p_values]

# Plot the entropy function
plt.figure(figsize=(10, 6))
plt.plot(p_values, entropy_values)
plt.axvline(x=0.5, color='r', linestyle='--', label='Maximum at p=0.5')
plt.xlabel('Probability p')
plt.ylabel('Entropy H(p)')
plt.title('Binary Entropy Function')
plt.grid(True)
plt.legend()
plt.show()
```

4.2 Mutual Information (8 points)

Assume a composer writes a melody by choosing for the first measure two notes from the 8 major scale notes spanning an octave, and for the second measure choosing the same two notes plus either the first or fifth scale degree. The purpose of the question is to consider how much information (bits) need to be sent if we already know something about how a sequence is generated, or how it may relate to another sequence (such as its own past)

1. If we treat the composer as an information source, and we know that they always use the above method for writing their melodies, how many bits are required to represent the first bar?

Hint: Determine how many bits are required to represent initially the alphabet (the total number of possible notes in octave), which gives you # of bits per note without any prior information or compressions.

2. How many bits are required to represent the second bar?

Hint: Consider the number of bits you need when you use the prior information from the first bar to encode the second bar.

3. If we represent the first bar as random variable X and the second bar as random variable Y , write the expressions for $H(X)$, $H(Y)$, and $H(Y|X)$.
4. What is the mutual information $I(X, Y)$ between the first and second bar? Hint: Think about how many bits of information are already contained in X in order to compose Y .

Setup:

- We have an 8-note major scale (an alphabet of size 8).

- **First measure (bar):** 2 notes chosen freely from the 8.
- **Second measure:** the *same* 2 notes from the first bar, plus **either** the 1st or the 5th scale degree (i.e., only 2 possible extra notes).
- WE ARE CONSIDERING CHOOSING WITH REPLACEMENT, WHICH ALIGN WELL WITH NATURE

1. Bits Required to Represent the First Bar

- Each note is chosen from 8 possibilities, requiring $\log_2(8) = 3$ bits per note.
- Because the first bar has 2 notes:

$$H(X) = 2 \times 3 = 6 \text{ bits.}$$

Hence, **6 bits** are needed to encode the first measure with no prior information.

2. Bits Required to Represent the Second Bar

- Once the first bar X is known, we already know which 2 notes are used in the second bar.
- The only "new" information is which extra note (the 1st or 5th scale degree) is added.
- There are 2 possibilities, so

$$\text{Information for the second bar given the first} = \log_2(2) = 1 \text{ bit.}$$

Hence, **1 bit** is needed to encode the second bar given the first bar.

3. Entropy Expressions

Let X = "first bar," Y = "second bar." Then:

- $H(X)$: the entropy of the first bar. We've seen that X consists of 2 notes from 8 possibilities each:

$$H(X) = \log_2(8^2) = 6 \text{ bits.}$$

- $H(Y)$: the entropy of the second bar *if viewed in isolation*.
 - The second bar has 3 notes. However, it is constrained so that the first 2 notes match *some* pair from 8×8 possibilities, and the 3rd note is one of 2 possibilities. Hence the total number of possible second bars is $8 \times 8 \times 2 = 128$.
 - So $H(Y) = \log_2(128) = 7$ bits.
- $H(Y | X)$: the conditional entropy of the second bar given the first bar.
 - Given X , the 2 notes in Y are fixed, and only the choice between scale degree 1 or 5 remains:

$$H(Y | X) = \log_2(2) = 1 \text{ bit.}$$

4. Mutual Information $I(X; Y)$

By definition,

$$I(X; Y) = H(Y) - H(Y | X).$$

From the above:

$$H(Y) = 7 \text{ bits}, \quad H(Y | X) = 1 \text{ bit},$$

so

$$I(X; Y) = 7 - 1 = 6 \text{ bits.}$$

Interpretation: Knowing X "saves" you 6 bits of description about Y , leaving only 1 bit of genuine "new" information (whether the composer added the 1st or 5th scale degree).

5 ELBO

5.1 Latent Variables (10 points)

1. The basic idea in the Variational Method and Expectation-Maximization is to maximize the ELBO instead of likelihood $P(X|\theta)$. With this "trick", we can find the optimal model shown in Class. For this problem we provide a slightly different formulation, where the expectation E is done over $q(Z)$ instead of $q(Z|x)$. Both versions of ELBO are used in practice, depending on whether the objective is to construct an encoder Q that is more sensitive to input X or not.

Prove that the log likelihood in $p(X|\theta)$ can be decomposed as $ELBO + KL(q||p)$, where

$$ELBO = \mathcal{L}(q, \theta) = \sum_z q(Z) \ln \frac{p(x, Z|\theta)}{q(Z)}$$

and

$$KL(q||p) = - \sum_z q(Z) \ln \frac{p(Z|x, \theta)}{q(Z)}$$

with q representing the distribution of the latent variables Z and $KL(q||p)$ representing the Kullback-Leibler divergence between the distributions q and p .

Hint: Use definitions of conditional probability (Bayes' rule) so that the sum of the ELBO and KL expressions cancel the dependency of the probability on the variable Z .

2. ELBO and EM are closely related. One is approximating the likelihood using VAE through differentiable programming (i.e. gradient descent), and the other is an iterative solution. While VAE uses gradient descent to find the approximate distribution q , the EM method uses an old estimate of $p(x|z, \theta)$ as the approximation.

In the EM algorithm, the expectation of the complete-data log likelihood evaluated for some general parameter value θ is given as

$$Q(\theta, \theta^{old}) = \sum_Z p(Z|X, \theta^{old}) \ln p(X, Z|\theta)$$

where $p(Z|X, \theta^{old})$ is the posterior distribution of the latent variables estimated in the E step using the current parameter values θ^{old} . Show that the ELBO is the same as $Q(\theta, \theta^{old})$, up to an entropy term in q for $q = p(z|x, \theta)$.

Hint: entropy is defined as $\sum q \ln q$.

3. Argue that the best approximate distribution q for a given parameter θ is the posterior distribution $p(Z|X, \theta)$, and use this argument to explain the purpose of the E step in the EM

algorithm.

4. Show that maximizing the ELBO is equivalent to maximizing $Q(\theta, \theta_{old})$. Hint: verify which part of the equation depends on the model parameter θ .

We have a latent variable model with observed variable X , latent variable Z , and parameters θ . The *variational approach* posits an auxiliary distribution $q(Z)$ over Z to approximate the true posterior $p(Z | X, \theta)$. We'll show how the log-likelihood $\ln p(X | \theta)$ can be decomposed into the **ELBO** (Evidence Lower BOund) plus a **KL divergence** term between q and the true posterior.

1. The Decomposition: $\ln p(X | \theta) = \text{ELBO} + \text{KL}$

We want to prove that

$$\ln p(X | \theta) = \underbrace{\sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)}}_{\text{ELBO}} + \underbrace{\left[- \sum_z q(Z) \ln \frac{p(Z | X, \theta)}{q(Z)} \right]}_{\text{KL divergence}},$$

or more succinctly,

$$\ln p(X | \theta) = \mathcal{L}(q, \theta) + \text{KL}(q \| p).$$

1. Write $\ln p(X | \theta)$ as a sum over Z

Since

$$p(X | \theta) = \sum_z p(X, Z | \theta),$$

we have

$$\ln p(X | \theta) = \ln \sum_z p(X, Z | \theta).$$

2. Multiply and Divide by $q(Z)$

Insert an arbitrary distribution $q(Z)$ inside the sum:

$$p(X, Z | \theta) = \frac{p(X, Z | \theta)}{q(Z)} q(Z).$$

Hence,

$$\ln p(X | \theta) = \ln \sum_z q(Z) \frac{p(X, Z | \theta)}{q(Z)}.$$

3. Apply Jensen's Inequality

Define $\mathbb{E}_{q(Z)}[\cdot] = \sum_z q(Z) [\cdot]$. Recognize that we have a log of an expectation:

$$\ln p(X | \theta) = \ln \mathbb{E}_{Z \sim q} \left[\frac{p(X, Z | \theta)}{q(Z)} \right].$$

By Jensen's inequality (using $\ln(\mathbb{E}[X]) \geq \mathbb{E}[\ln X]$), we get

$$\ln p(X | \theta) \geq \sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)} =: \mathcal{L}(q, \theta),$$

which is the **ELBO** (Evidence Lower Bound).

4. Rearrange and Identify KL Divergence

Finally, consider the *difference* between $\ln p(X | \theta)$ and $\mathcal{L}(q, \theta)$. By expanding $\ln p(X, Z | \theta) = \ln p(X | \theta) + \ln p(Z | X, \theta)$ and grouping terms, one finds:

$$\ln p(X | \theta) - \sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)} = \sum_z q(Z) \ln \frac{q(Z)}{p(Z | X, \theta)}.$$

The right-hand side is exactly the definition of the Kullback–Leibler divergence:

$$\text{KL}(q(Z) \| p(Z | X, \theta)) = \sum_z q(Z) \ln \frac{q(Z)}{p(Z | X, \theta)}.$$

Hence,

$$\ln p(X | \theta) = \underbrace{\sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)}}_{\text{ELBO}} + \underbrace{\text{KL}(q(Z) \| p(Z | X, \theta))}_{\geq 0}.$$

Thus we obtain the **ELBO + KL** decomposition:

$$\boxed{\ln p(X | \theta) = \mathcal{L}(q, \theta) + \text{KL}(q(Z) \| p(Z | X, \theta))}.$$

Since KL divergence is always non-negative, this shows $\mathcal{L}(q, \theta)$ is indeed a **lower bound** on $\ln p(X | \theta)$.

2. Relation to EM and the Function $Q(\theta, \theta^{old})$

EM's $Q(\theta, \theta^{old})$ expression

In the EM algorithm, for a current parameter estimate θ^{old} , the *E step* computes the posterior $p(Z | X, \theta^{old})$, and then one defines

$$Q(\theta, \theta^{old}) = \sum_Z p(Z | X, \theta^{old}) \ln p(X, Z | \theta).$$

Let's compare this to the ELBO in the special case that $q(Z) = p(Z | X, \theta^{old})$. Then the ELBO is

$$\mathcal{L}(q, \theta) = \sum_Z p(Z | X, \theta^{old}) \ln \frac{p(X, Z | \theta)}{p(Z | X, \theta^{old})}.$$

Observe that

$$\mathcal{L}(q, \theta) = \sum_Z p(Z | X, \theta^{old}) \ln p(X, Z | \theta) - \sum_Z p(Z | X, \theta^{old}) \ln p(Z | X, \theta^{old}).$$

The second term $-\sum_Z p(Z | X, \theta^{old}) \ln p(Z | X, \theta^{old})$ is the *entropy* of the distribution $p(Z | X, \theta^{old})$. Hence:

$$\mathcal{L}(p(Z | X, \theta^{old}), \theta) = Q(\theta, \theta^{old}) - H(p(Z | X, \theta^{old})).$$

So **ELBO** differs from **Q-function** by an additive constant $-H(p(Z | X, \theta^{old}))$ (which does not depend on θ). Hence maximizing $\mathcal{L}$ w.r.t. θ is equivalent to maximizing $Q(\theta, \theta^{old})$.

3. Best Approximate Distribution q

From the decomposition

$$\ln p(X | \theta) = \underbrace{\sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)}}_{\mathcal{L}(q, \theta)} + \text{KL}(q(Z) \| p(Z | X, \theta)),$$

note that $\ln p(X | \theta)$ does **not** depend on q . Therefore, minimizing the KL divergence term $\text{KL}(q \| p(Z | X, \theta))$ w.r.t. q is achieved precisely when

$$q(Z) = p(Z | X, \theta).$$

Thus, **the “best” approximate distribution** under this framework is the true posterior.

Interpretation in EM:

- The E-step effectively sets $q(Z) = p(Z | X, \theta^{old})$.
- This ensures we’re using the posterior over Z given the most recent parameter estimate θ^{old} .

That is exactly how EM updates the distribution of latent variables: in the E-step, we compute or approximate the posterior with the current parameters.

4. Maximizing ELBO $\iff$ Maximizing $Q(\theta, \theta^{old})$

Recall that

$$\mathcal{L}(p(Z | X, \theta^{old}), \theta) = Q(\theta, \theta^{old}) - H(p(Z | X, \theta^{old})).$$

Because the **entropy** term $H(p(Z | X, \theta^{old}))$ does **not** depend on the new parameters θ , the objective $\mathcal{L}$ and $Q(\theta, \theta^{old})$ differ by a θ -independent constant. Therefore, maximizing the ELBO w.r.t. θ is the same as maximizing $Q(\theta, \theta^{old})$.

Conclusion: Once $q(Z) = p(Z | X, \theta^{old})$ is chosen, any parameter θ that increases $\mathcal{L}(q, \theta)$ equivalently increases $Q(\theta, \theta^{old})$. Hence the M-step is exactly maximizing $\mathcal{L}$ or Q .

Summary of Answers

1. ELBO + KL Decomposition

$$\ln p(X | \theta) = \underbrace{\sum_z q(Z) \ln \frac{p(X, Z | \theta)}{q(Z)}}_{\text{ELBO}} + \underbrace{\text{KL}(q(Z) \| p(Z | X, \theta))}_{\geq 0}.$$

2. Relation to EM’s $Q(\theta, \theta^{old})$

For $q(Z) = p(Z | X, \theta^{old})$,

$$\mathcal{L}(q, \theta) = Q(\theta, \theta^{\text{old}}) - H(p(Z | X, \theta^{\text{old}})).$$

3. Best q is the posterior

Minimizing $\text{KL}(q \| p(Z | X, \theta))$ forces $q(Z) = p(Z | X, \theta)$. EM's E-step uses the old parameters to obtain the posterior over Z .

4. Maximizing ELBO $\iff$ Maximizing $Q(\theta, \theta^{\text{old}})$

Because they differ only by a θ -independent entropy constant, improving θ to maximize the ELBO also maximizes $Q(\theta, \theta^{\text{old}})$.

Thus, the EM algorithm can be viewed as an alternating maximization of the ELBO: first pick $q = p(Z | X, \theta^{\text{old}})$ (E-step), then maximize over θ (M-step).

6 Gaussian Approximation (4 points)

Suppose that $p(x)$ is some fixed distribution and that we wish to approximate it using a Gaussian distribution $g(x) = N(x | \mu, \Sigma)$. By writing down the form of the KL divergence $\text{KL}(p \| g)$ for a Gaussian $g(x)$ and then differentiating, show that minimization of $\text{KL}(p \| g)$ with respect to μ and Σ leads to the result that μ is given by the expectation of x under $p(x)$ and that Σ is given by the covariance.

1. Express the KL Divergence $\text{KL}(p \| g)$

The KL divergence between p and g is

$$\text{KL}(p \| g) = \int p(x) \ln \frac{p(x)}{g(x)} dx = \int p(x) \ln p(x) dx - \int p(x) \ln g(x) dx.$$

Since $\int p(x) \ln p(x) dx$ does **not** depend on μ or Σ , minimizing $\text{KL}(p \| g)$ is equivalent to **maximizing** the second term $\int p(x) \ln g(x) dx$, or equivalently **minimizing** the negative of that term.

So we focus on the part:

$$- \int p(x) \ln g(x) dx.$$

2. Write Down $\ln g(x)$ for the Gaussian

A multivariate Gaussian with mean μ and covariance Σ has density

$$g(x) = \frac{1}{\sqrt{(2\pi)^D \det(\Sigma)}} \exp\left[-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right].$$

Hence,

$$\ln g(x) = -\frac{D}{2} \ln(2\pi) - \frac{1}{2} \ln \det(\Sigma) - \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu),$$

where D is the dimension of x .

Therefore,

$$- \int p(x) \ln g(x) dx = \int p(x) \left[\frac{D}{2} \ln(2\pi) + \frac{1}{2} \ln \det(\Sigma) + \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu) \right] dx.$$

We can factor out constant terms, etc.

3. Differentiate w.r.t. μ and Σ

3.1 Minimize w.r.t. μ

Focus on the part that depends on μ :

$$\frac{1}{2} \int p(x) (x - \mu)^\top \Sigma^{-1} (x - \mu) dx.$$

We can rewrite $(x - \mu)^\top \Sigma^{-1} (x - \mu)$ as $\text{tr}[\Sigma^{-1} (x - \mu)(x - \mu)^\top]$. For convenience, think of the derivative in simpler scalar forms:

- Set derivative w.r.t. μ to zero.
- Expand:

$$(x - \mu)^\top \Sigma^{-1} (x - \mu) = \text{const} - 2 (\Sigma^{-1} x)^\top \mu + \mu^\top \Sigma^{-1} \mu.$$

Hence, if we define

$$\mathcal{L}(\mu, \Sigma) = \int p(x) [(x - \mu)^\top \Sigma^{-1} (x - \mu)] dx,$$

then differentiating w.r.t. μ (treating Σ^{-1} as constant for this partial derivative) yields:

$$\frac{\partial \mathcal{L}}{\partial \mu} = -2 \Sigma^{-1} \int p(x) x dx + 2 \Sigma^{-1} \mu \int p(x) dx = -2 \Sigma^{-1} \mathbb{E}_p[x] + 2 \Sigma^{-1} \mu.$$

Setting this equal to zero implies

$$\Sigma^{-1} \mu = \Sigma^{-1} \mathbb{E}_p[x] \implies \mu = \mathbb{E}_p[x].$$

Thus the optimal μ is the **mean** of x under p .

3.2 Minimize w.r.t. Σ

Now set $\mu = \mathbb{E}_p[x]$ and handle the terms involving Σ . We have two pieces that depend on Σ in the negative log-likelihood portion:

1. $\frac{1}{2} \ln \det(\Sigma)$
2. $\frac{1}{2} \int p(x) (x - \mu)^\top \Sigma^{-1} (x - \mu) dx$

Hence, the relevant function of Σ to minimize is (omitting constants):

$$\mathcal{F}(\Sigma) = \frac{1}{2} \ln \det(\Sigma) + \frac{1}{2} \int p(x) (x - \mu)^\top \Sigma^{-1} (x - \mu) dx.$$

Let's denote $S = \mathbb{E}_p[(x - \mu)(x - \mu)^\top]$, the covariance of p . Then

$$\mathcal{F}(\Sigma) = \frac{1}{2} \ln \det(\Sigma) + \frac{1}{2} \text{tr}[\Sigma^{-1} S].$$

Taking the derivative w.r.t. Σ^{-1} is standard in matrix calculus. The known results are:

- $\frac{\partial}{\partial \Sigma^{-1}} \ln \det(\Sigma) = -(\Sigma)^{\top} = -\Sigma$ (since Σ is symmetric).
- $\frac{\partial}{\partial \Sigma^{-1}} \text{tr}(\Sigma^{-1}S) = -S^{\top} = -S$ (also using the symmetry of S).

Hence,

$$\frac{\partial \mathcal{F}(\Sigma)}{\partial \Sigma^{-1}} = -\frac{1}{2} \Sigma + \frac{1}{2} S.$$

Setting this to zero gives

$$\Sigma = S = \mathbb{E}_p[(x - \mu)(x - \mu)^{\top}].$$

Thus Σ must be the **covariance** of p .

4. Conclusion

By minimizing $\text{KL}(p(x) \parallel \mathcal{N}(x \mid \mu, \Sigma))$ with respect to μ and Σ , we get:

$$\boxed{\mu = \mathbb{E}_p[x], \quad \Sigma = \mathbb{E}_p[(x - \mu)(x - \mu)^{\top}].}$$

That is, we match the mean and covariance of the Gaussian to the mean and covariance of p .

This completes the proof.

7 Probabilistic PCA

In probabilistic PCA, the observations are assumed to be generated from latent variable z with added noise ϵ according to

$$x = Wz + \mu + \epsilon$$

1. Using expressions for the mean and covariance of x , prove that

$$E[x] = \mu$$

and

$$\text{cov}[x] = WW^T + \sigma$$

In your explanation, use the assumption that

$$z \sim \mathcal{N}(\mathbf{0}, I)$$

Explain where you use this assumption.

2. It can be shown that:

$$p(z|x) = \mathcal{N}(z \mid M^{-1}W^T(x - \mu), M^{-1}\sigma^{-2})$$

where M is defined as

$$M = W^TW + \sigma^2 I$$

Show that as $\sigma \rightarrow 0$, the posterior mean for z given x becomes

$$E(z|x) = (W^TW)^{-1}W^T(x - \mu)$$

3. In non-probabilistic (regular) PCA, the goal is to approximate a vector x that has D dimensions by combination from a smaller set of basis vectors $w_1, w_2, \dots, w_M$ with $M < D$. Arranging the basis vectors as columns of a matrix W , consider $\hat{x} = Wz$ to be a low dimensional approximation to x . Explain why z is M -dimensional, and show that for a fixed W , the optimal z is found by the pseudo-inverse of W given by

$$(W^T W)^{-1} W^T$$

Compare this to the maximum likelihood result for probabilistic PCA, and describe your observation.

Hint: to find a pseudo-inverse, write $x = Wz + \text{error}$. Then, write an expression for MSE and minimize with respect to z , showing that optimal z is given by $(W^T W)^{-1} W^T x$.

1. Mean and Covariance of x

We want to prove:

1. $E[x] = \mu$.
2. $\text{cov}[x] = W W^T + \sigma^2 I$.

1.1 Compute $E[x]$

By definition,

$$x = Wz + \mu + \epsilon.$$

Taking the expectation:

$$\begin{aligned} E[x] &= E[Wz] + E[\mu] + E[\epsilon] \\ &= W E[z] + \mu + E[\epsilon]. \end{aligned}$$

We know:

- $z \sim \mathcal{N}(0, I)$ implies $E[z] = 0$.
- μ is a constant vector, so $E[\mu] = \mu$.

We used the assumption $z \sim \mathcal{N}(0, I)$ to conclude $E[z] = 0$. Why did we assume so?

- We assume z is zero-mean and identity-covariance so that all variance structure in the observed data arises from W and the noise covariance $\sigma^2 I$.
- Mathematically, it makes the derivation neat and yields closed-form solutions for the posterior and MLE updates.
- Conceptually, it aligns perfectly with the interpretation of PCA as finding orthonormal directions of maximal variance, captured in W .

Hence, and by another hidden assumption that $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$:

$$E[x] = W \cdot 0 + \mu + 0 = \mu.$$

1.2 Compute $\text{cov}[x]$

Next, we write:

$$\text{cov}[x] = \mathbb{E}[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^\top].$$

Since $\mathbb{E}[x] = \mu$, we have

$$x - \mu = (Wz + \mu + \epsilon) - \mu = Wz + \epsilon.$$

Thus,

$$\text{cov}[x] = \mathbb{E}[(Wz + \epsilon)(Wz + \epsilon)^\top] = \mathbb{E}[Wz z^\top W^\top] + \mathbb{E}[\epsilon \epsilon^\top] + \mathbb{E}[Wz \epsilon^\top] + \mathbb{E}[\epsilon z^\top W^\top].$$

Because z and ϵ are assumed **independent** (that's part of the generative model assumption), we get

$$\mathbb{E}[Wz \epsilon^\top] = W \mathbb{E}[z] \mathbb{E}[\epsilon]^\top = 0,$$

and similarly $\epsilon z^\top = 0$. Therefore,

$$\text{cov}[x] = W \mathbb{E}[z z^\top] W^\top + \mathbb{E}[\epsilon \epsilon^\top].$$

Given $z \sim \mathcal{N}(0, I)$, we have $\mathbb{E}[z z^\top] = I$. Also, $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ implies $\mathbb{E}[\epsilon \epsilon^\top] = \sigma^2 I$. Hence,

$$\text{cov}[x] = W I W^\top + \sigma^2 I = W W^\top + \sigma^2 I.$$

2. Posterior of z Given x

It is known that:

$$p(z | x) = \mathcal{N}\left(z \mid M^{-1} W^\top (x - \mu), M^{-1} \sigma^{-2}\right),$$

where

$$M = W^\top W + \sigma^2 I.$$

We want to show that as $\sigma \rightarrow 0$, the posterior mean

$$\mathbb{E}[z | x] = M^{-1} W^\top (x - \mu)$$

reduces to

$$(W^\top W)^{-1} W^\top (x - \mu).$$

2.1 As $\sigma \rightarrow 0$

Recall

$$M = W^\top W + \sigma^2 I \implies M^{-1} = (W^\top W + \sigma^2 I)^{-1}.$$

When σ is very small, $\sigma^2 I \approx 0$. Hence in the limit $\sigma \rightarrow 0$,

$$M^{-1} \rightarrow (W^\top W)^{-1}.$$

Thus,

$$\mathbb{E}[z | x] = M^{-1} W^\top (x - \mu) \rightarrow (W^\top W)^{-1} W^\top (x - \mu).$$

Hence, the posterior mean becomes precisely the *least-squares solution* for z , consistent with classical PCA.

3. Connection to Non-Probabilistic (Classical) PCA

In **non-probabilistic PCA**, we think:

1. $x \in \mathbb{R}^D$ is approximated by $\hat{x} = Wz$ where $W \in \mathbb{R}^{D \times M}$.
2. Because we want a "low dimensional" representation, we have $M < D$. So $z \in \mathbb{R}^M$.

3.1 Why is z M -dimensional?

- We choose M basis vectors $\{w_1, w_2, \dots, w_M\}$. Each $w_i \in \mathbb{R}^D$.
- Arranging these as columns in W gives a $D \times M$ matrix.
- If we want a linear combination $Wz = \sum_{i=1}^M z_i w_i$, then z must have M components.

Hence dimension of z is exactly the number of columns in W .

3.2 Optimal z for Fixed W

Given a data vector x , the classical PCA approximation is

$$x \approx Wz \implies \text{error} = x - Wz.$$

We define a mean-square error (MSE) objective:

$$\|x - Wz\|^2,$$

and want to pick z that **minimizes** this quantity, for a **fixed** W . Hence,

$$\min_z \|x - Wz\|^2 = \min_z (x - Wz)^\top (x - Wz).$$

To find the optimum, **set derivative to zero**:

$$\frac{\partial}{\partial z} [(x - Wz)^\top (x - Wz)] = -2 W^\top (x - Wz) = 0.$$

Solving,

$$W^\top x - W^\top Wz = 0 \implies W^\top Wz = W^\top x \implies z = (W^\top W)^{-1} W^\top x,$$

assuming $W^\top W$ is invertible (i.e. W has full column rank).

Thus the **pseudo-inverse** of W is $(W^\top W)^{-1} W^\top$.

3.3 Compare to Probabilistic PCA

From **Section 2**, the **posterior mean** for the latent variable z given x (in the noise-free limit $\sigma \rightarrow 0$) is:

$$\mathbb{E}[z | x] = (W^\top W)^{-1} W^\top (x - \mu).$$

Hence in the probabilistic PCA framework, once we subtract the mean μ , we get the same formula as the classical PCA pseudo-inverse solution.

Observation:

- Probabilistic PCA can be seen as a probabilistic version of the classical approach.
- In the limit of zero noise $\sigma \rightarrow 0$, the posterior mean of z coincides with the least squares solution $\hat{z}$ from classical PCA.
- However, in the **probabilistic** version, we also get a full posterior **distribution** $p(z | x)$, not just a point estimate.

Summary of Answers

1. Mean and Covariance of x :

$$\mathbb{E}[x] = \mu \quad \text{and} \quad \text{cov}[x] = W W^\top + \sigma^2 I,$$

using the assumptions $z \sim \mathcal{N}(0, I)$ and $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

2. Posterior Mean: $p(z | x)$ is Gaussian with

$$\mathbb{E}[z | x] = (W^\top W + \sigma^2 I)^{-1} W^\top (x - \mu).$$

As $\sigma \rightarrow 0$,

$$\mathbb{E}[z | x] \rightarrow (W^\top W)^{-1} W^\top (x - \mu).$$

3. Connection to Classical PCA:

- In non-probabilistic PCA, we minimize $\|x - Wz\|^2$.
- The optimal z w.r.t. MSE is $(W^\top W)^{-1} W^\top x$.
- This matches the **noise-free** ($\sigma = 0$) posterior mean solution in probabilistic PCA.

Hence, **probabilistic PCA** generalizes classical PCA by placing a Gaussian prior on z and adding Gaussian noise ϵ . The EM algorithm or direct maximum-likelihood in the probabilistic framework yields the same principal subspace solutions, but also a richer interpretation in terms of latent-variable distributions.

Part III

DKU Assignment 1

PCA with Linear Autoencoder

Instructions:

- This notebook is an interactive assignment; please read and follow the instructions in each cell.
- Cells that require your input (in the form of code or written response) will have 'Question #' above.
- After completing the assignment, please submit this notebook as a PDF.
- Make sure to mark the page with your solution for each problem on Gradescope. Any problems without the correct pages marked may receive a score of 0.

```
In [23]: import torch
import torch.nn as nn
from torch.autograd import Variable
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.stats import truncnorm
```

PCA with a Linear Autoencoder

In this problem, we will practice using basic neural network procedures by running an autoencoder network. This network is implemented in pyTorch.

Let's create a sin+noise signal to use as input to our network:

```
In [27]: f0 = 200
fs = 10000
T = 3
Ns = fs*T

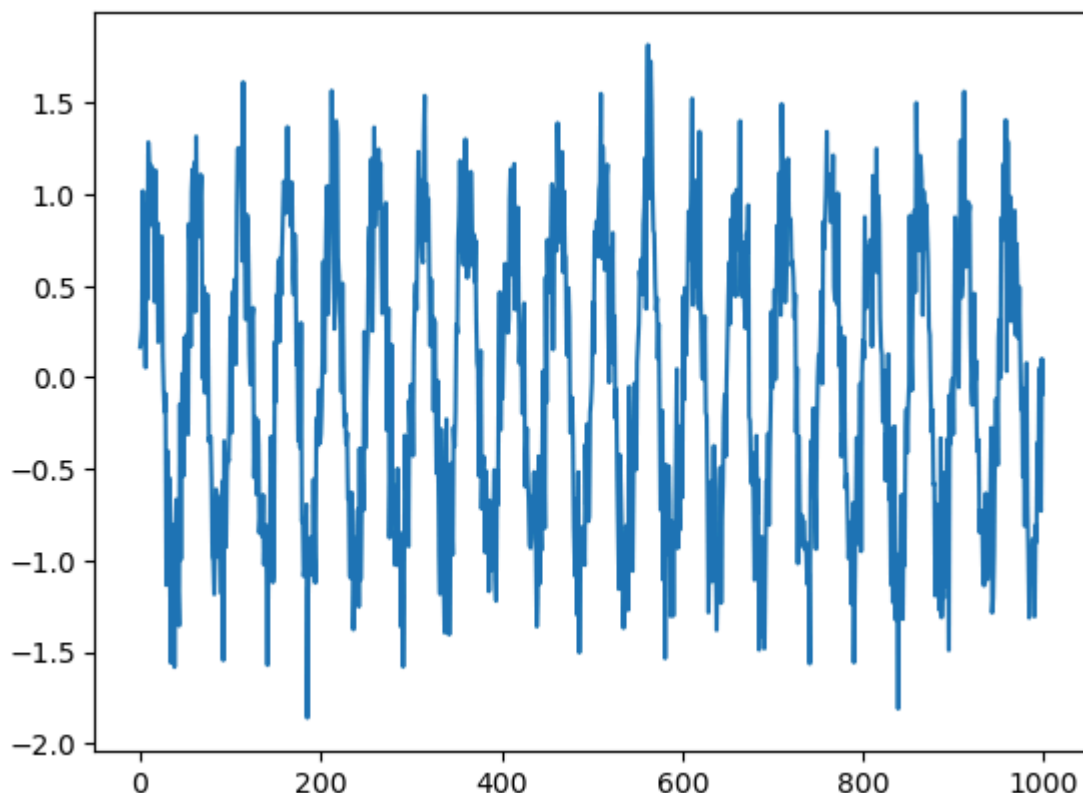
def buffer(input_data, sample_rate, window_size, hop_size):
    output = np.array([input_data[i:i+window_size] for i in range(0, len(input_data)-
        return output.T

s = np.sin(2*np.pi*f0*np.arange(Ns)/fs)

n = np.random.randn(Ns)

x = s + 0.3*n

plt.plot(x[:1000])
wavfile.write('out2.wav', fs, x)
xmat = buffer(x,fs,400,200)
```



Here we create an AE with 2 hidden layers.

This neural network is implemented in pyTorch.

Please review the code cells below, and answer the questions that follow.

```
In [28]: n_inputs = np.shape(xmat)[0]
n_hidden = 2
learning_rate = 0.01

W=Variable(torch.normal(0, 0.1, (400,2)), requires_grad=True)

optimizer=torch.optim.Adam([W],lr=learning_rate)
```

```
In [29]: n_iterations = 10000
X_train = torch.tensor(xmat.T,dtype=torch.float32)
hidden=torch.matmul(X_train,W)
X_test = X_train
codings = hidden

col = ['b','r','g','c','m','y','k']

for iteration in range(n_iterations):
    hidden=torch.matmul(X_train,W)
    outputs=torch.matmul(hidden,torch.transpose(W,0,1))
    reconstruction_loss=torch.mean(torch.square(outputs-X_train))

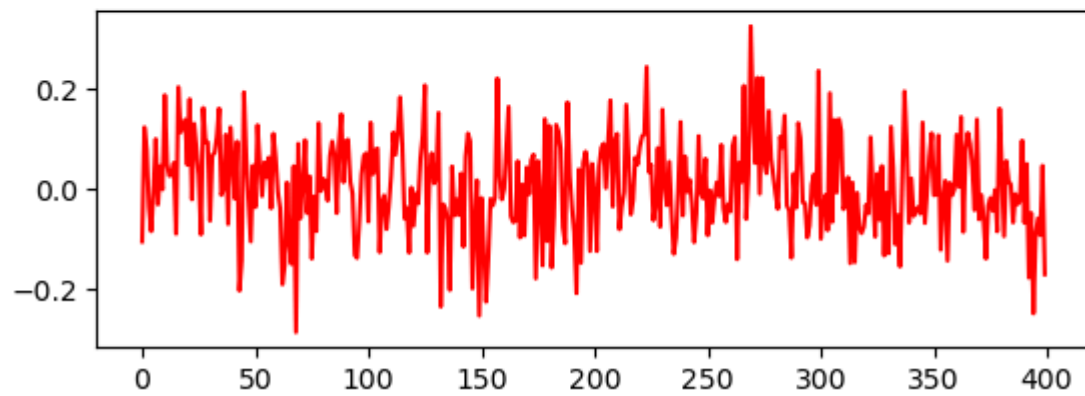
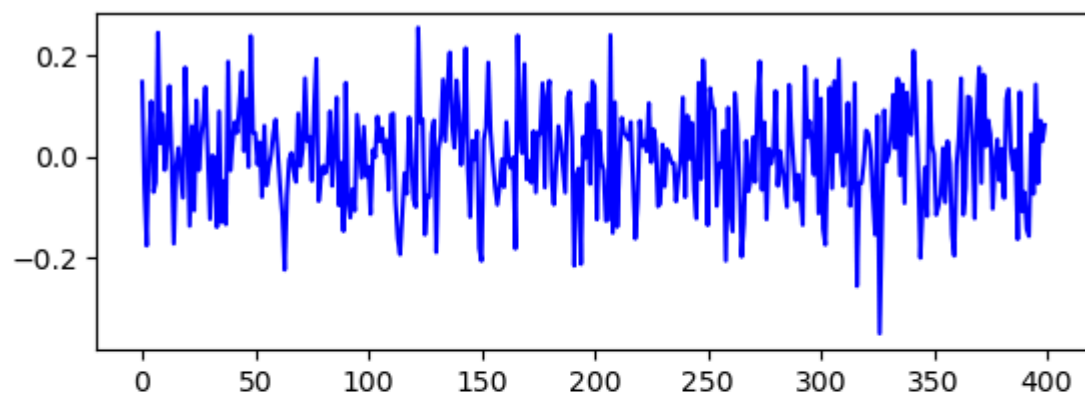
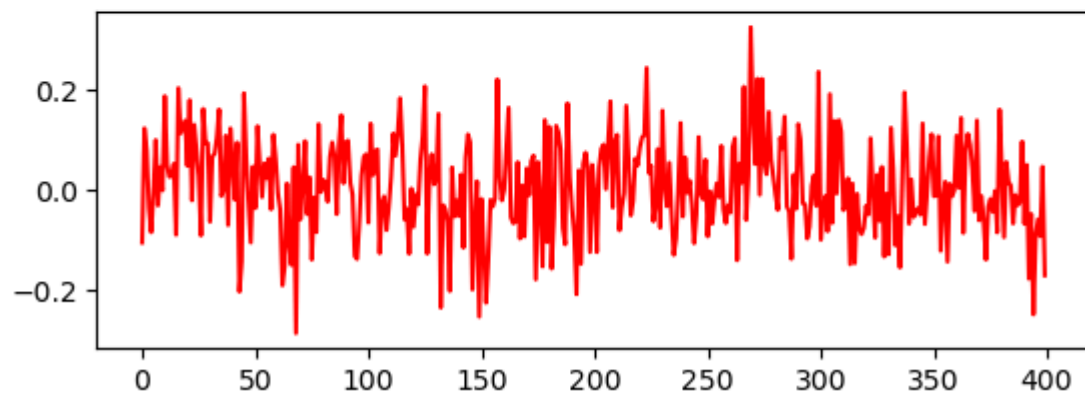
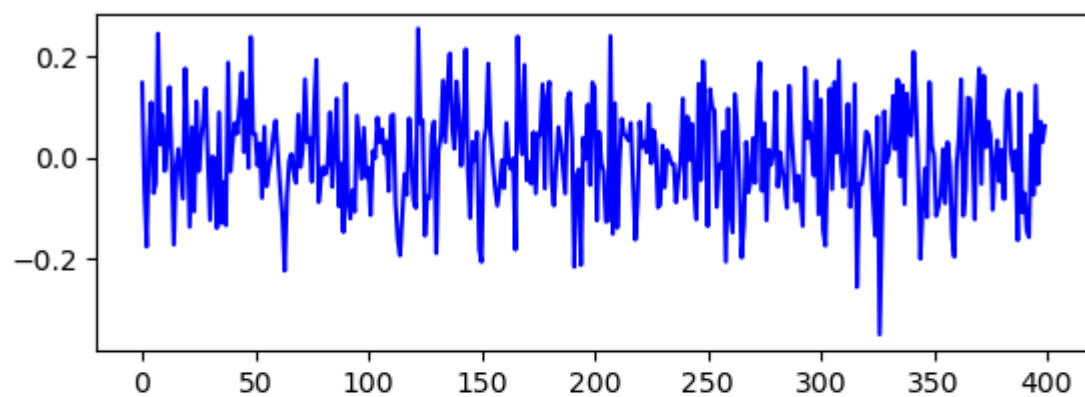
    optimizer.zero_grad()
    reconstruction_loss.backward()
    optimizer.step()

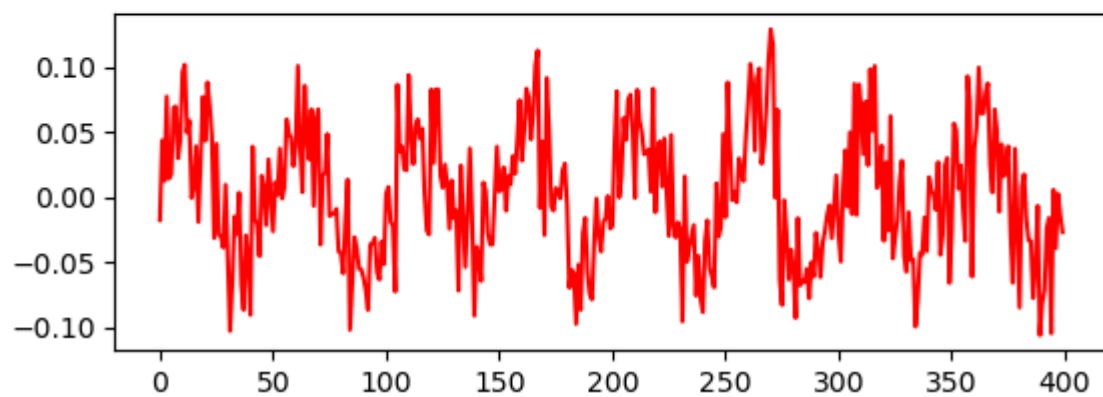
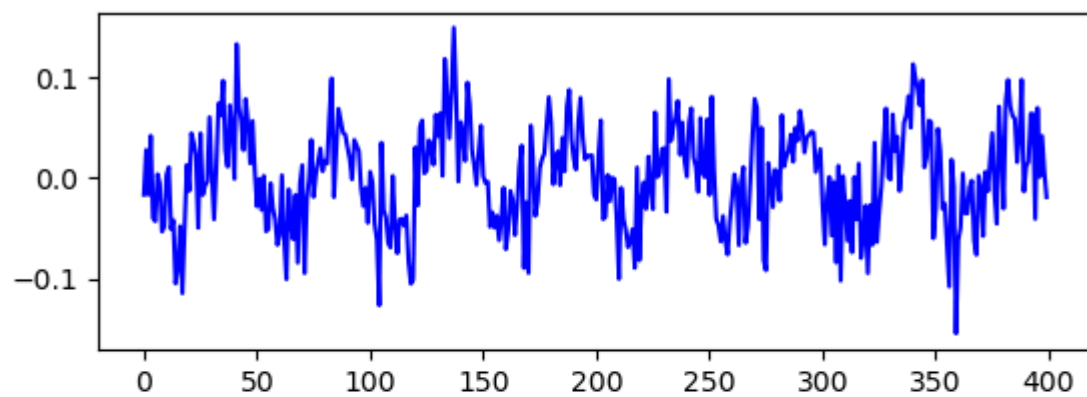
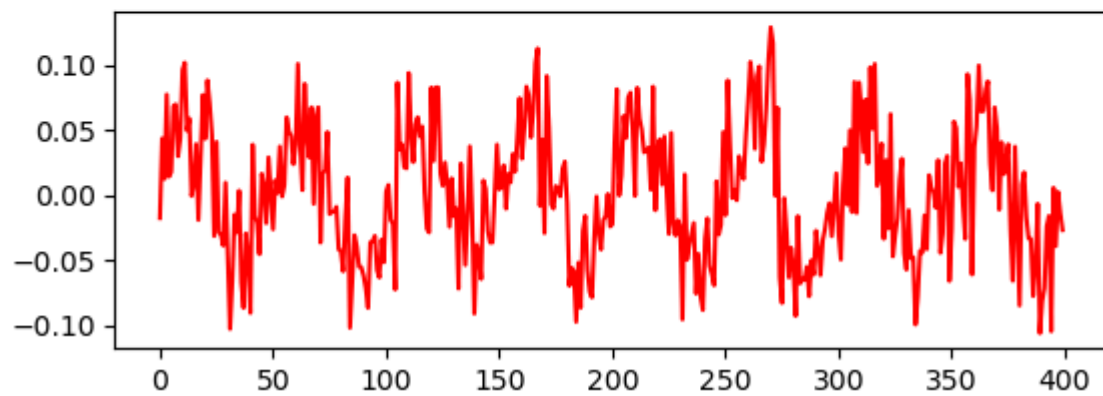
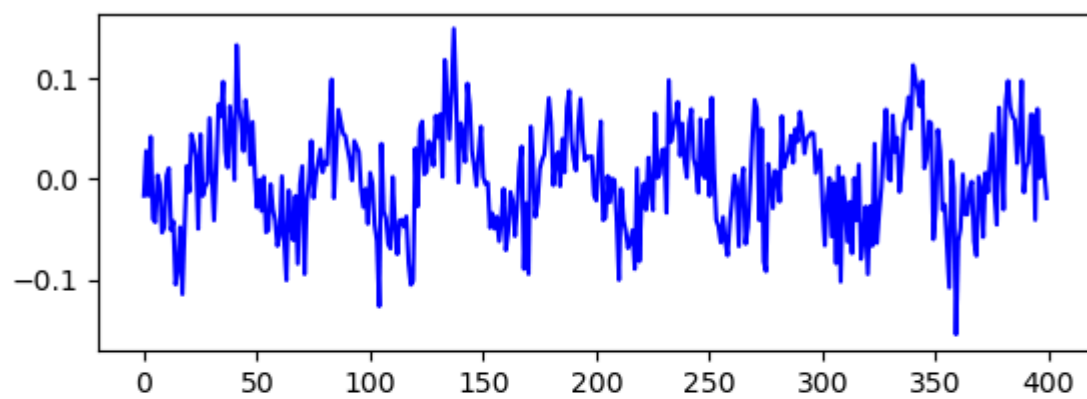
    if iteration%1000==0:
        plt.clf()
        for k in range(n_hidden):
            plt.subplot(n_hidden,1,k+1)
            plt.plot(W.detach().numpy()[ :,k],col[k % len(col)])
        plt.show(False)
        plt.pause(0.001)

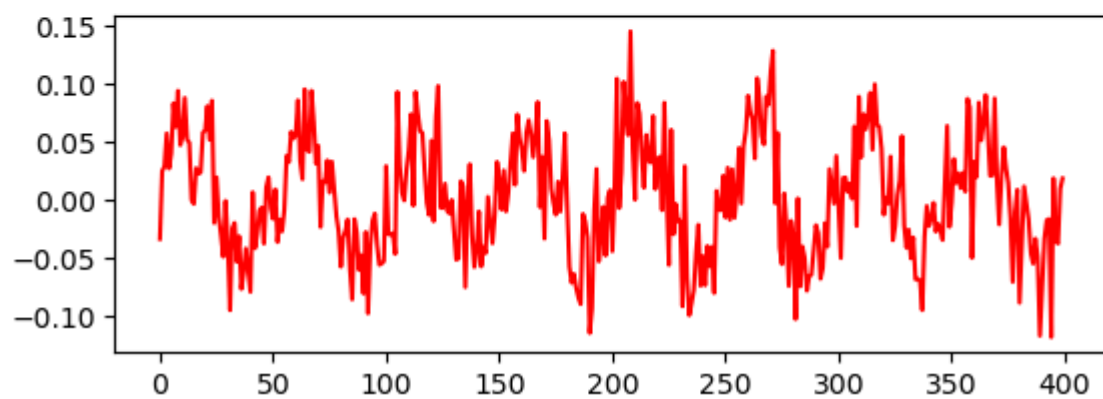
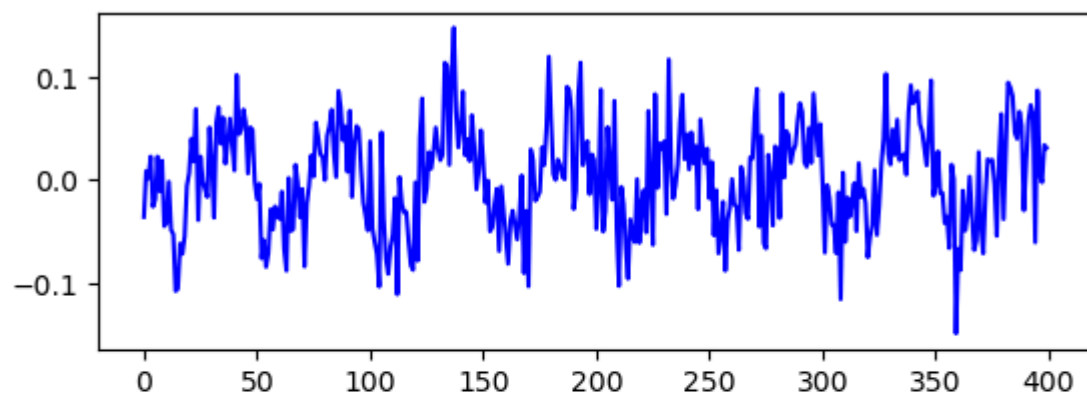
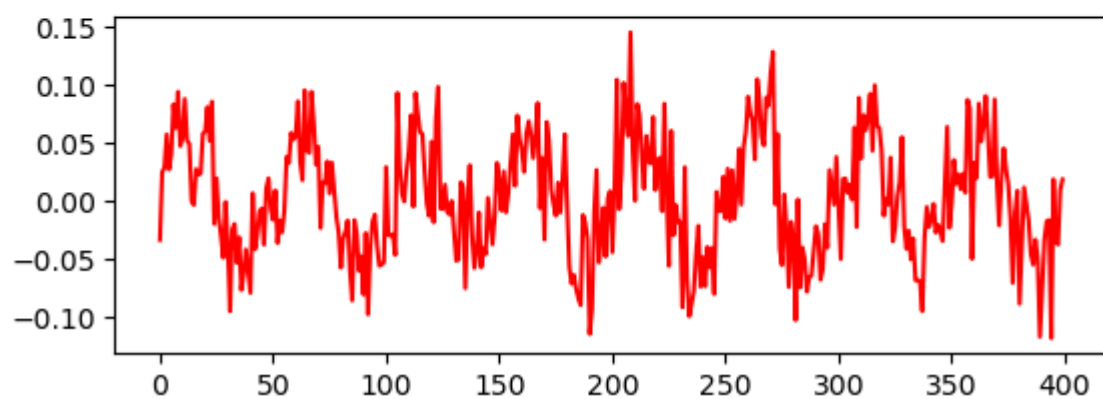
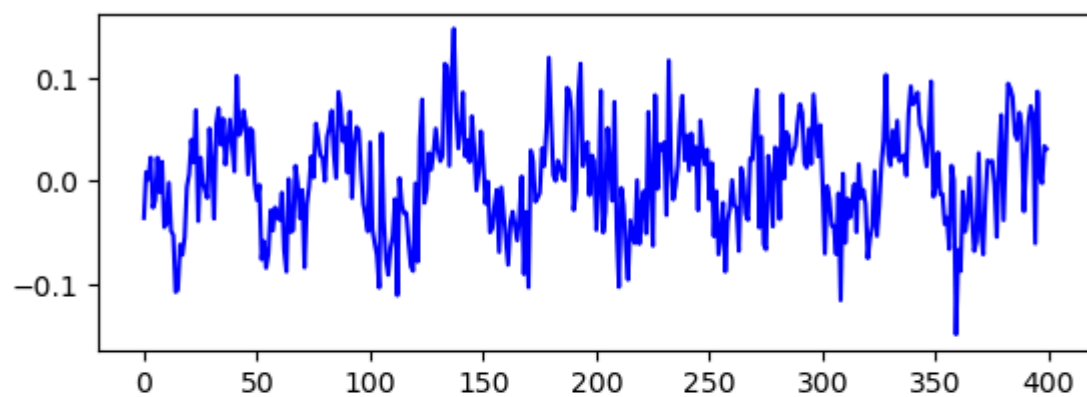
    codings=torch.matmul(X_test,W)
    outputs_eval=torch.matmul(codings,torch.transpose(W,0,1))
    reconstruction_loss_eval=torch.mean(torch.square(outputs_eval-X_test))

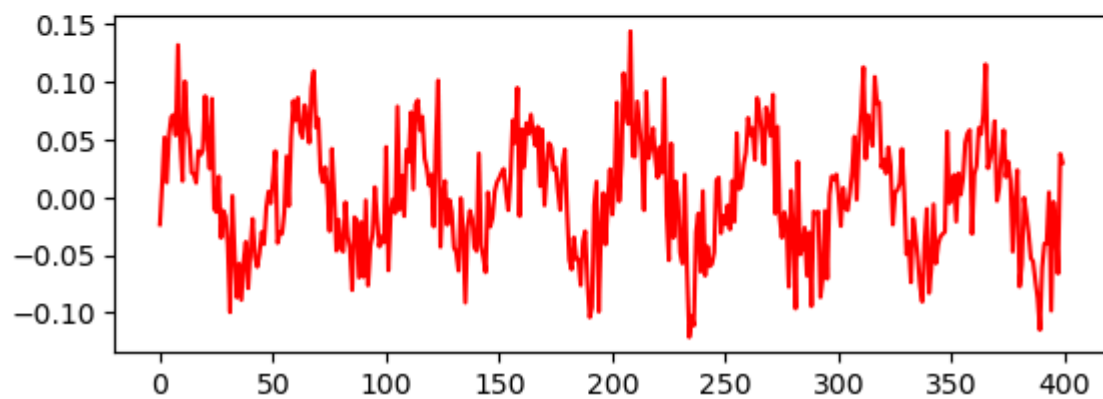
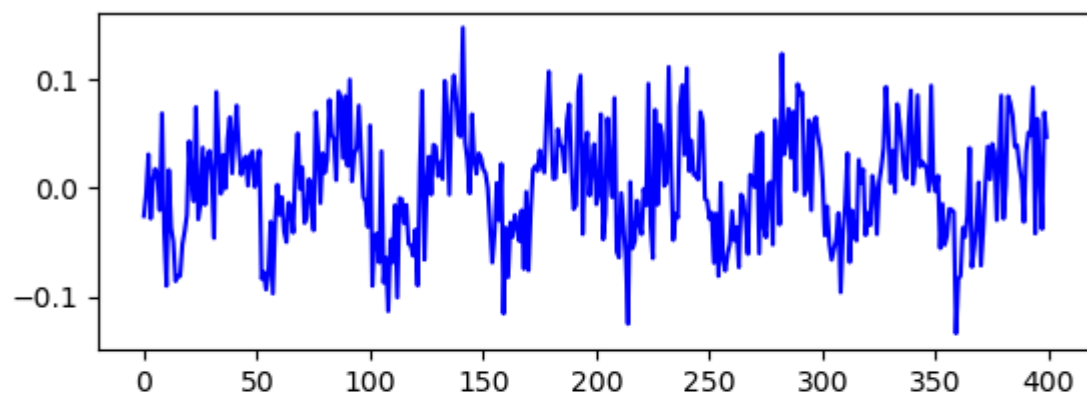
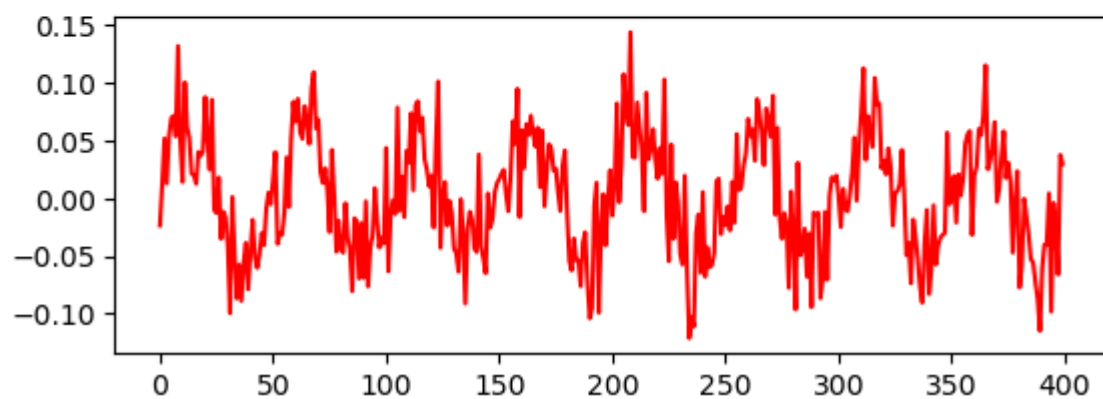
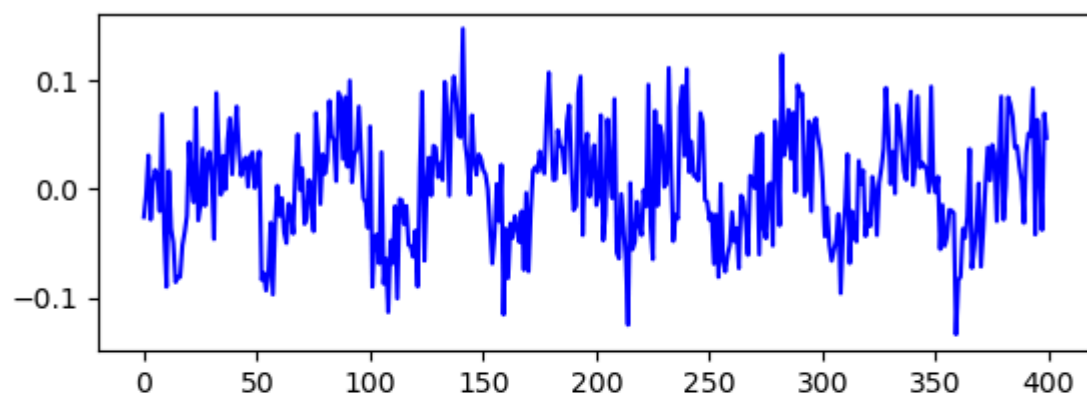
    codings_val = codings.detach().numpy()

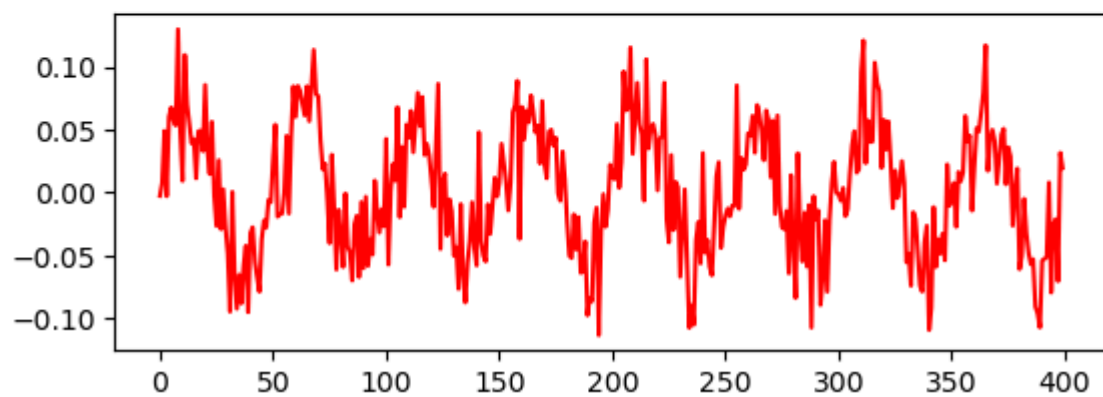
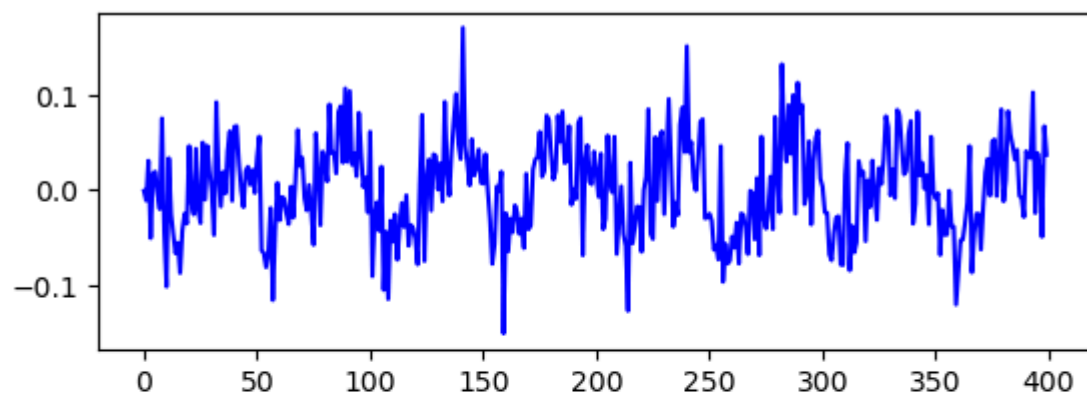
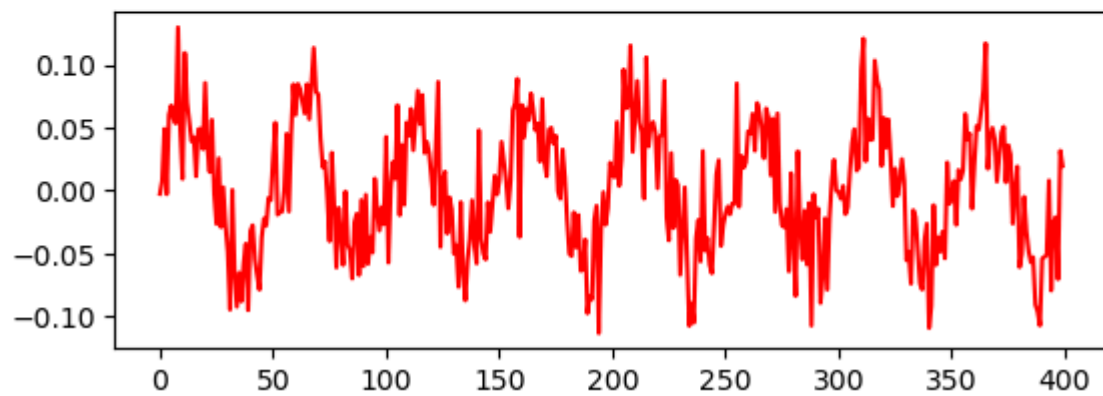
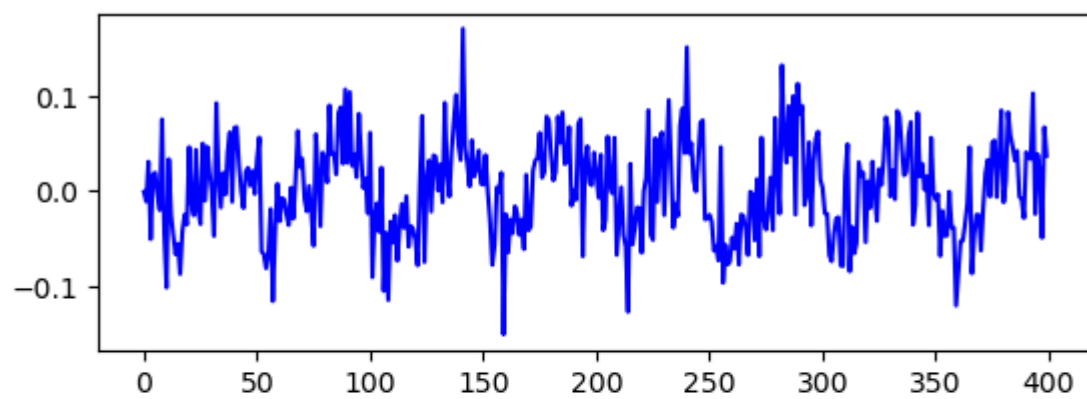
print("Done with training")
```

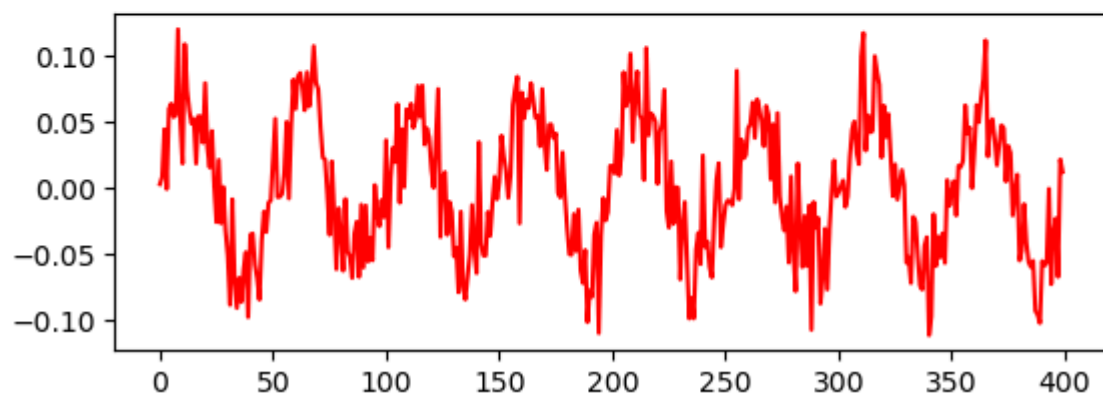
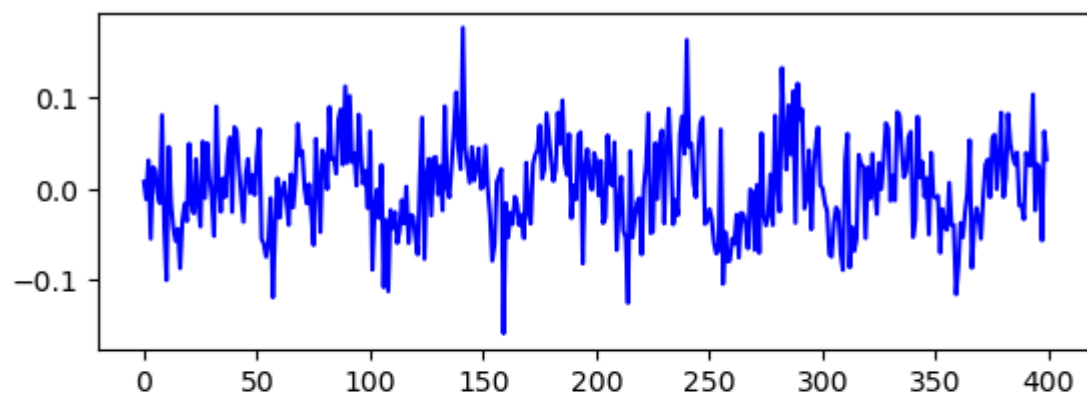
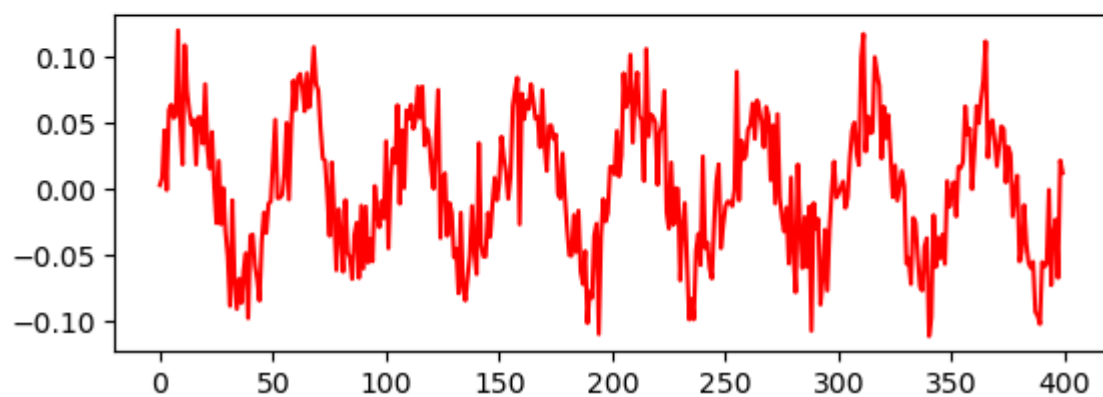
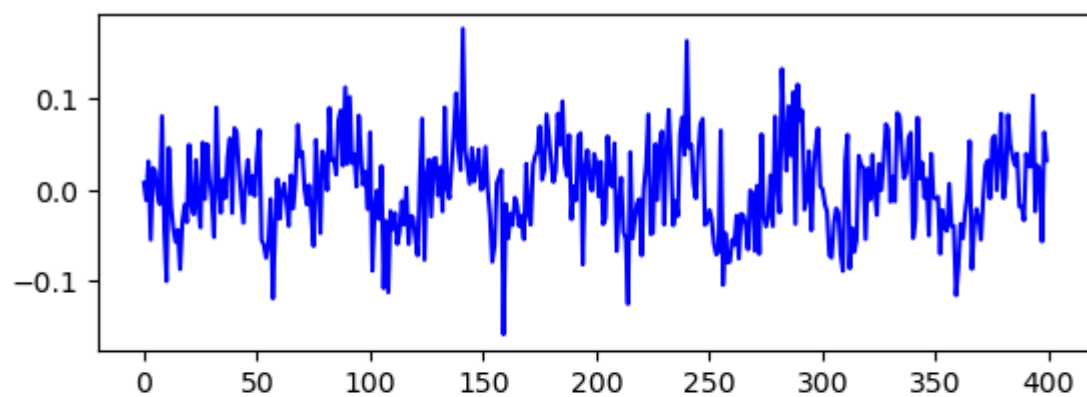


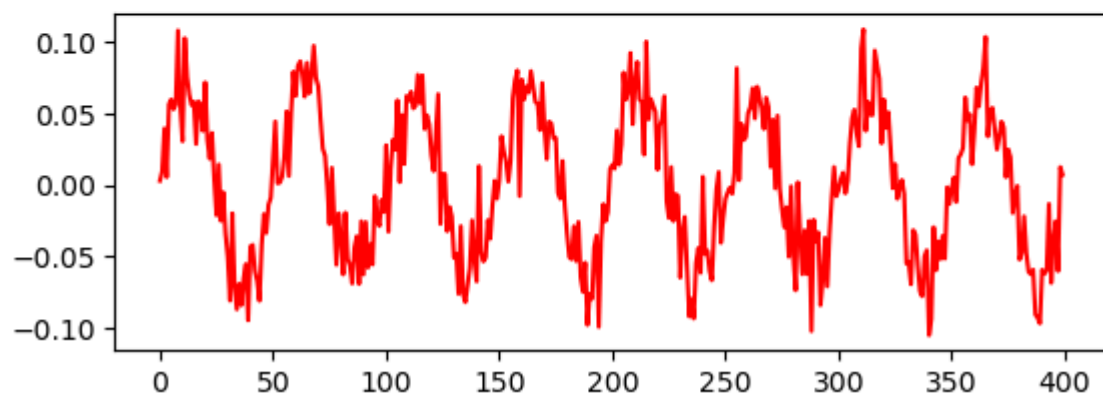
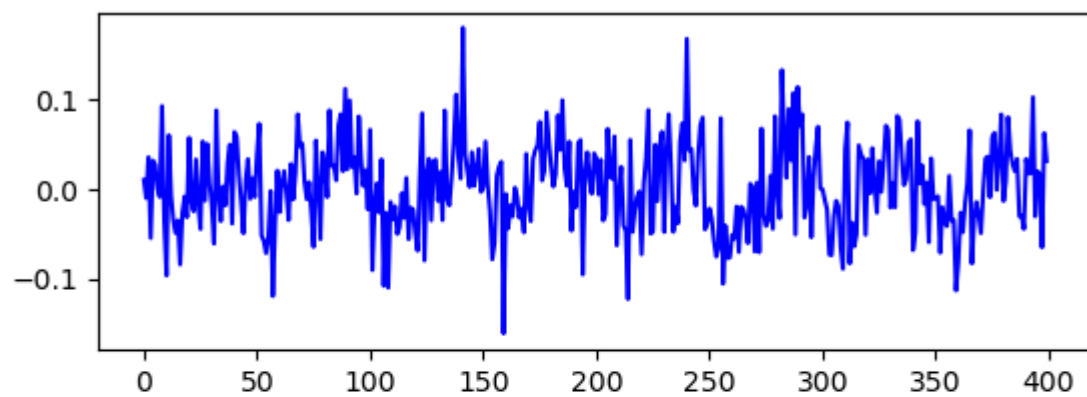
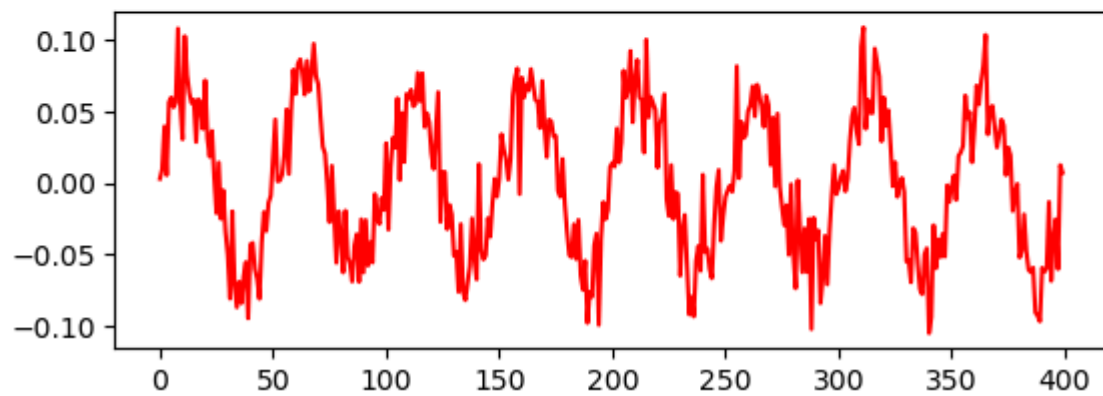
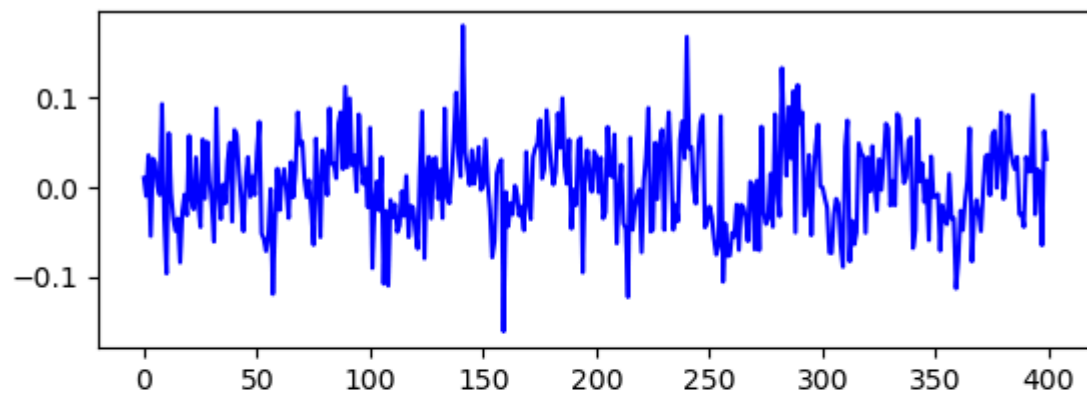


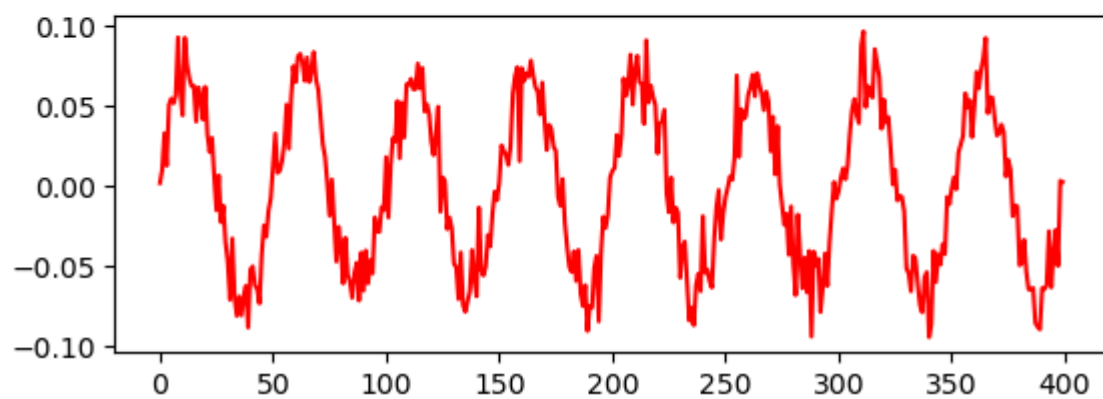
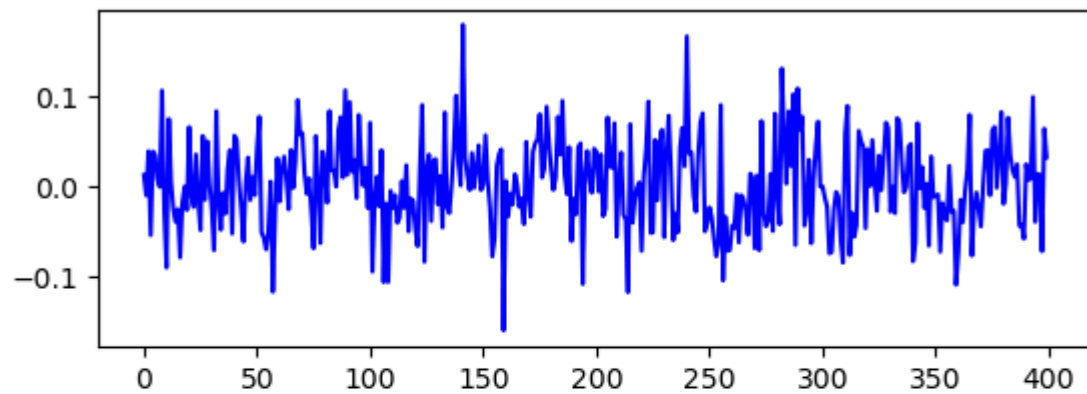
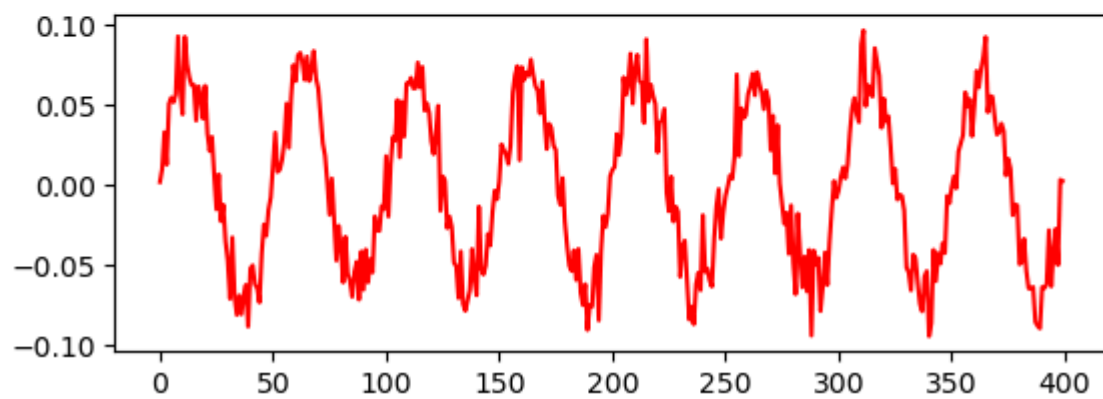
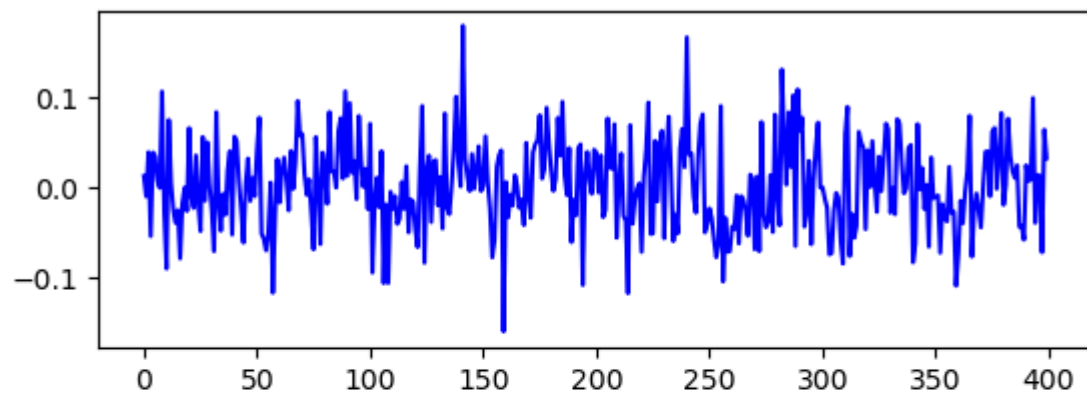


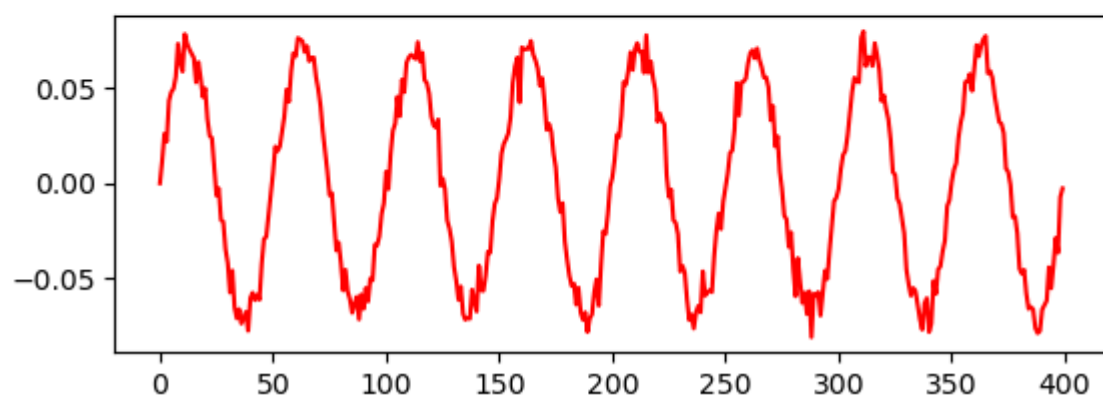
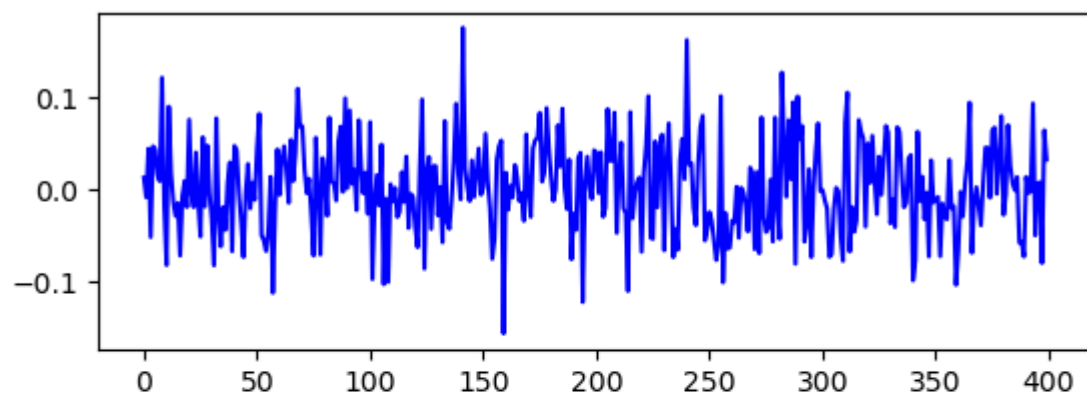
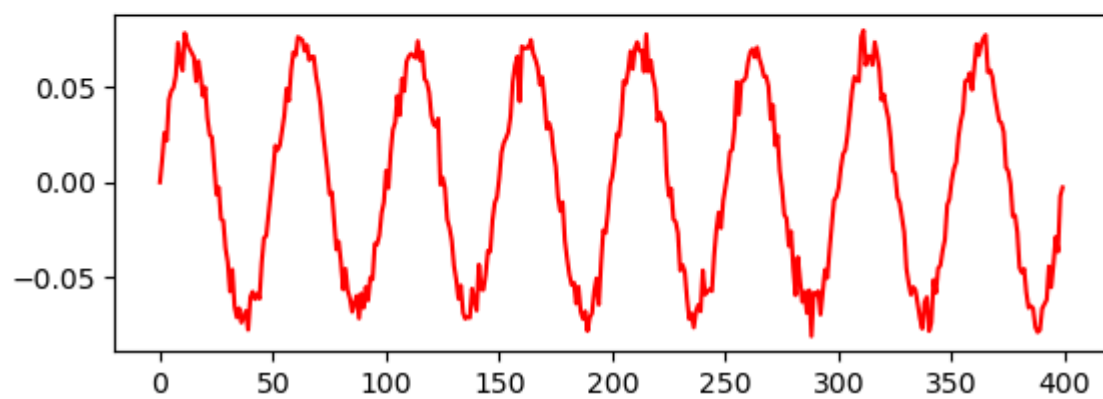
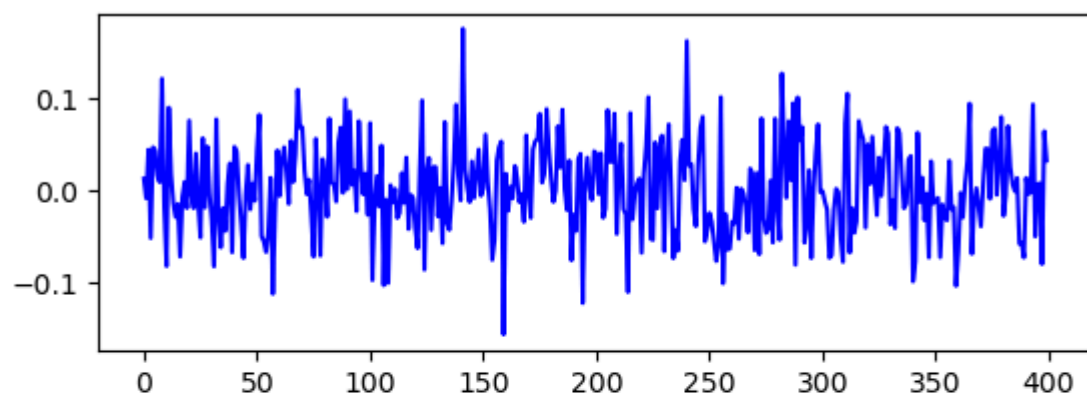


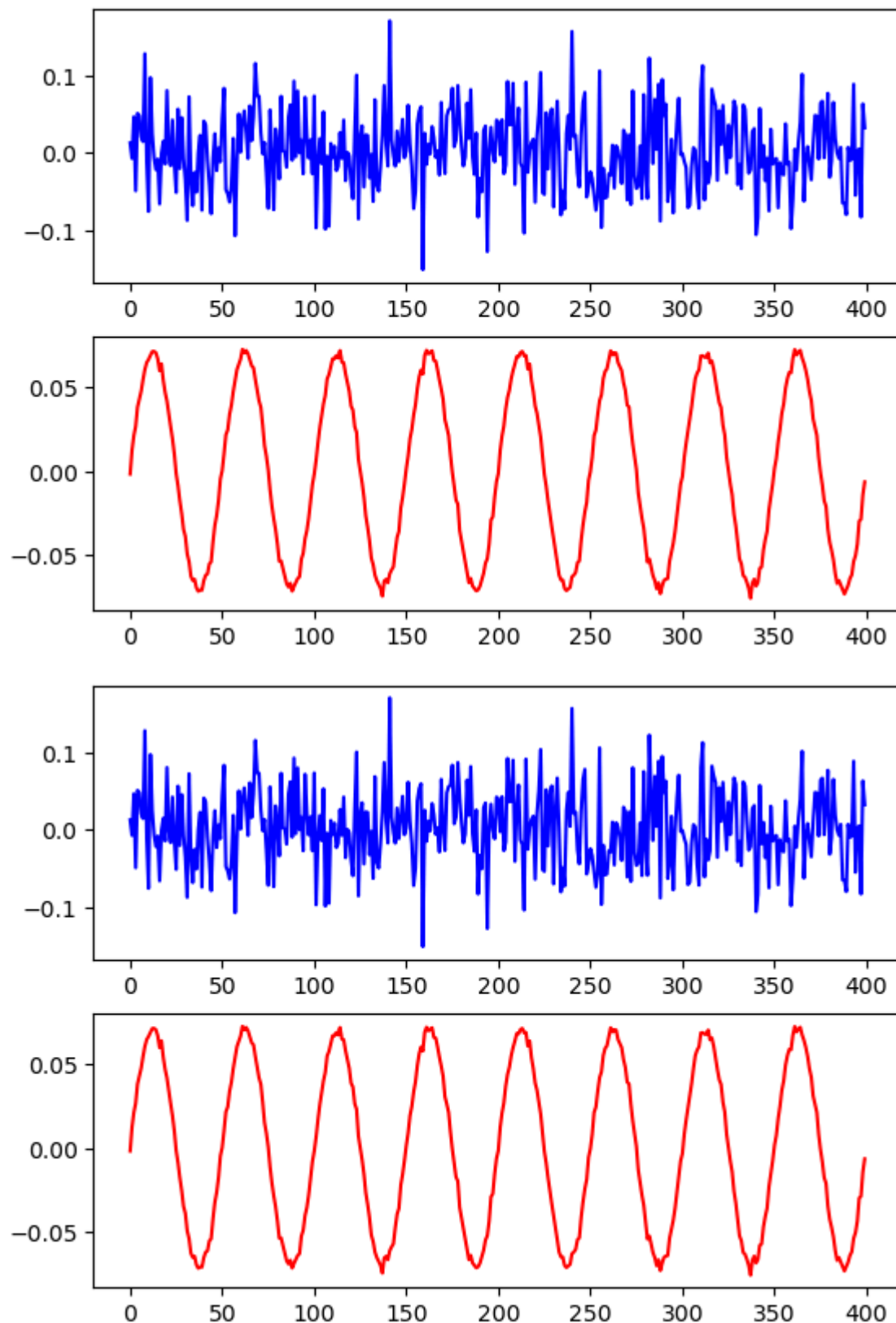












Done with training

Question 1 (5 points)

What is an autoencoder? Please explain briefly. What would happen (ideally) if you pass a portion of signal x through the trained network?

Answer:

An autoencoder is a type of neural network that learns to encode input data into a lower-dimensional representation (encoding) and then decode it back to reconstruct the original input. It consists of two main parts: an encoder that compresses the input into a latent-space representation, and a decoder that reconstructs the input from this representation.

In this implementation, the encoder is represented by the matrix multiplication of X_{train} with W , creating the "hidden" or "codings" variable, while the decoder is represented by multiplying these codings with the transpose of W .

Ideally, if we pass a portion of signal x through the trained network, the network would first encode it into a lower-dimensional representation (in this case, 2 dimensions as defined by $n_{\text{hidden}} = 2$), and then decode it back to reconstruct the original signal with minimal loss of information. The reconstructed signal would preserve the most important features of the original signal while filtering out noise, effectively acting as a denoising mechanism for the sinusoidal signal contaminated with random noise.

Question 2 (5 points)

Based on the observed shape of n_{inputs} and the definition of X_{train} , what exactly is being passed to the input layer of the network for a single forward pass? Be specific!

Answer:

Based on the code, $n_{\text{inputs}} = \text{np.shape}(\text{xmat})[0]$ which is 400, and $X_{\text{train}} = \text{torch.tensor}(\text{xmat.T}, \text{dtype}=\text{torch.float32})$.

The variable xmat is created by buffering the audio signal x using the buffer function, which creates a matrix where each column represents a window of the audio signal. The buffer function uses a window size of 400 samples and a hop size of 200 samples.

When xmat is transposed (xmat.T) to create X_{train} , each row of X_{train} represents one window of the audio signal. Therefore, in a single forward pass, the network receives a batch of these windows, where each window is a 400-dimensional vector representing a segment of the audio signal (a mixture of sine wave and noise).

The dimensions of X_{train} are [number of windows, 400], and each forward pass processes all these windows simultaneously through the weight matrix W of shape [400, 2].

Question 3 (5 points)

What variable(s) are used to represent the network weights? How are these weights initialized prior to training?

Answer:

The network weights are represented by the variable W , which is initialized as a `torch.Variable` with `requires_grad=True` to enable gradient computation during backpropagation. The weights are initialized using a normal distribution with mean 0 and standard deviation 0.1, creating a matrix of shape (400, 2) where 400 is the input dimension (window size) and 2 is the number of hidden units (n_{hidden}). This initialization is done with the line: `W=Variable(torch.normal(0, 0.1, (400,2)), requires_grad=True)`.

Question 4 (5 points)

What is being minimized in the reconstruction loss? Why is this helpful?

Answer:

The reconstruction loss minimizes the mean squared error between the original input data (X_{train}) and the reconstructed output (outputs). Specifically, it calculates the average of the squared differences between each element of the original input and its corresponding reconstructed value.

This is helpful because:

1. It forces the network to learn an efficient encoding of the data in the lower-dimensional space ($n_{\text{hidden}} = 2$)
2. Since our input signal contains a sine wave with noise, minimizing reconstruction loss helps the network learn to represent the underlying sine wave pattern while filtering out random noise
3. It enables unsupervised learning - no labels are needed, just the input data itself
4. The lower-dimensional representation (codings) can reveal the intrinsic structure of the data, potentially separating the sine wave components from noise

Question 5 (5 points)

What is an optimizer? What are 3 common optimizers? Which optimizer is used in this AE training?

Answer:

An optimizer is an algorithm used to adjust the parameters (weights and biases) of a neural network during training to minimize the loss function. It determines how the model's parameters are updated based on the gradients computed during backpropagation.

Three common optimizers in deep learning are:

1. **Stochastic Gradient Descent (SGD)**: The simplest optimizer that updates parameters in the opposite direction of the gradient with a fixed learning rate. It can be enhanced with momentum to accelerate convergence.
2. **Adam (Adaptive Moment Estimation)**: Combines the benefits of AdaGrad and RMSProp by maintaining adaptive learning rates for each parameter using both first-order (mean) and second-order (uncentered variance) moments of the gradients.
3. **RMSProp (Root Mean Square Propagation)**: Maintains a moving average of squared gradients for each parameter and divides the learning rate by the square root of this average, helping to handle different scales of gradients.
4. **AdamW**: Combines the benefits of Adam and weight decay regularization. It uses Adam's adaptive learning rates but adds a weight decay term to the update equation.

In this autoencoder training, the **Adam optimizer** is used, as seen in the code:

Question 6 (5 points)

Why is X_{test} set to X_{train} ?

Answer:

In this autoencoder implementation, X_{test} is set equal to X_{train} ($X_{\text{test}} = X_{\text{train}}$) because the model is being evaluated on the same data it was trained on. This means we're not using a separate test set to evaluate the model's generalization capabilities.

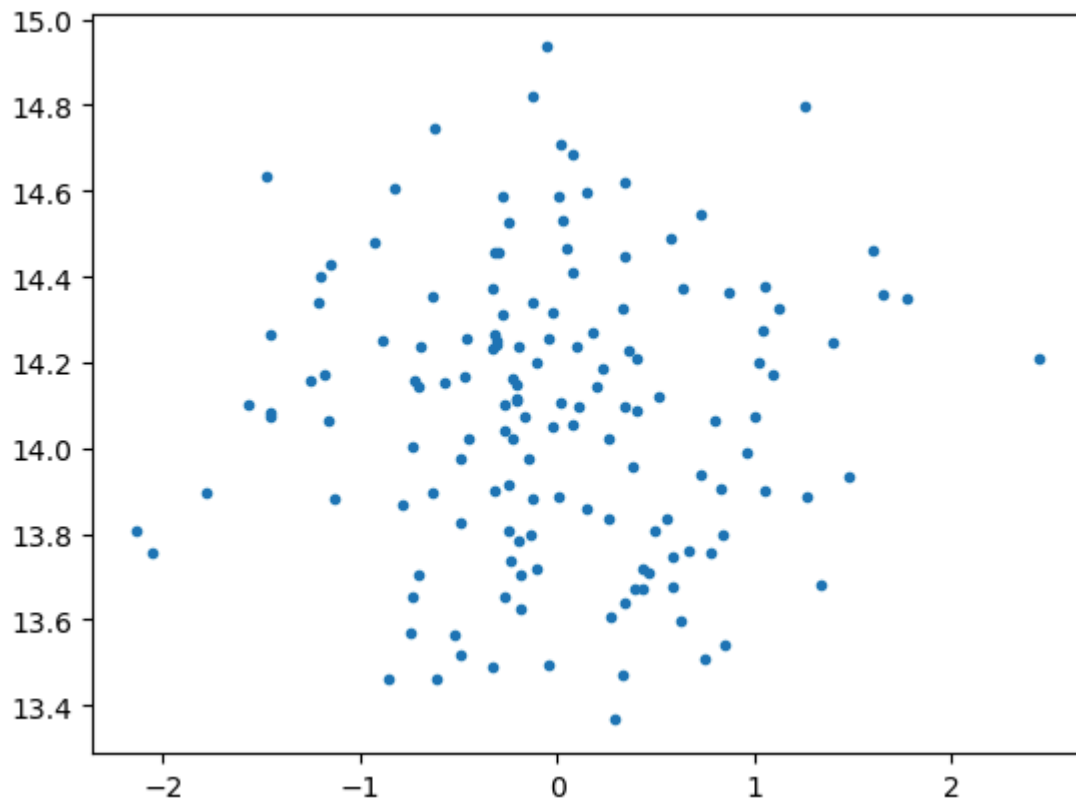
This approach has important implications for evaluation:

1. We're measuring the reconstruction ability on the training data itself, which tells us how well the model can compress and reconstruct the specific patterns it was trained to recognize.
2. We're not evaluating how well the model generalizes to unseen data, which would typically be done with a separate test set.
3. This is common in unsupervised learning scenarios like autoencoders where the goal is to learn efficient representations of the input data rather than to make predictions on new data.
4. The `reconstruction_loss_eval` metric calculated using `X_test` is essentially measuring the same thing as the training loss, just at a different point in the training loop.

We can examine now the "codings", i.e. the hidden unit values and their distribution. The more significant codings should have smaller variances.

```
In [30]: plt.plot(codings_val[:,0],codings_val[:,1],'.')
print ("mean: ", np.mean(codings_val,0))
print ("variance", np.std(codings_val,0))
```

```
mean: [-0.02060536 14.0797    ]
variance [0.77603126 0.3312787 ]
```



Question 7 (5 points)

In what way does the autoencoder network function similarly to PCA?

Answer:

The autoencoder network functions similarly to PCA in several key ways:

1. Dimensionality reduction: Both techniques compress high-dimensional data into a lower-dimensional representation (latent space) while preserving essential information.
2. Information preservation: Both prioritize capturing the most important variations in the data. PCA explicitly maximizes variance, while autoencoders minimize reconstruction error.
3. Linear transformations: In a single-layer autoencoder with linear activation functions, the learned representation is mathematically equivalent to PCA. The weight matrices of the encoder

effectively learn the principal component directions.

4. Data reconstruction: Both methods allow reconstruction of the original data from the compressed representation, with some information loss that increases as the dimensionality of the latent space decreases.
5. Unsupervised learning: Both techniques learn patterns from unlabeled data without requiring explicit target outputs.

The key difference is that autoencoders can learn non-linear transformations through multiple layers and non-linear activation functions, potentially capturing more complex patterns than PCA's strictly linear transformations.

Extra Credit (10 points)

Using `sklearn.decomposition.PCA` (or another PCA implementation in Python), illustrate the ideas you discuss in Question 1 & Question 7. You should have two plots: one showing the output of the AE network on a single sample, and another showing the output of your PCA on the same sample. How many principal components do you use in your reconstruction to achieve similar performance to the AE network?

```
In [34]: # Import necessary libraries
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# Select a single sample for comparison
sample_idx = 0
single_sample = X_test[sample_idx:sample_idx+1]
single_sample_np = single_sample.numpy().flatten()

# Get the autoencoder reconstruction for this sample
codings = torch.matmul(single_sample, W)
ae_reconstruction = torch.matmul(codings, torch.transpose(W, 0, 1))
ae_reconstruction = ae_reconstruction.detach().numpy().flatten()

# Calculate autoencoder error
ae_error = ((single_sample_np - ae_reconstruction)**2).mean()

# Fit PCA with maximum number of components we'll need
max_components = 20
pca = PCA(n_components=max_components)
pca.fit(X_train)

# Create figure for comparing reconstructions with different PCA components
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Original time series
axes[0, 0].plot(single_sample_np)
axes[0, 0].set_title('Original Time Series')
axes[0, 0].set_ylabel('Amplitude')

# Autoencoder reconstruction
axes[0, 1].plot(ae_reconstruction)
axes[0, 1].set_title(f'Autoencoder Reconstruction\nMSE: {ae_error:.4f}')

# PCA reconstructions with different numbers of components
pca_components = [1, 2, 3, 4]
pca_errors = []

# Transform the sample to PCA space
```

```

pca_transformed = pca.transform(single_sample_np.reshape(1, -1))

for i, n_comp in enumerate(pca_components):
    # Determine subplot position
    row, col = (0, 2) if i == 0 else (1, i-1)

    # Reconstruct using only n_comp components
    # Since PCA is deterministic, we can just use the first n_comp components
    truncated_transformed = np.copy(pca_transformed)
    truncated_transformed[0, n_comp:] = 0 # Zero out components we don't want to use
    pca_reconstruction = pca.inverse_transform(truncated_transformed).flatten()

    # Calculate error
    error = ((single_sample_np - pca_reconstruction)**2).mean()
    pca_errors.append(error)

    # Plot reconstruction
    axes[row, col].plot(pca_reconstruction)
    axes[row, col].set_title(f'PCA (n={n_comp}) Reconstruction\nMSE: {error:.4f}')

    # Add labels
    if col == 0:
        axes[row, col].set_ylabel('Amplitude')
    if row == 1:
        axes[row, col].set_xlabel('Time')

plt.tight_layout()
plt.show()

# Calculate errors for all component counts we want to analyze
n_components_range = range(1, max_components + 1)
all_pca_errors = []

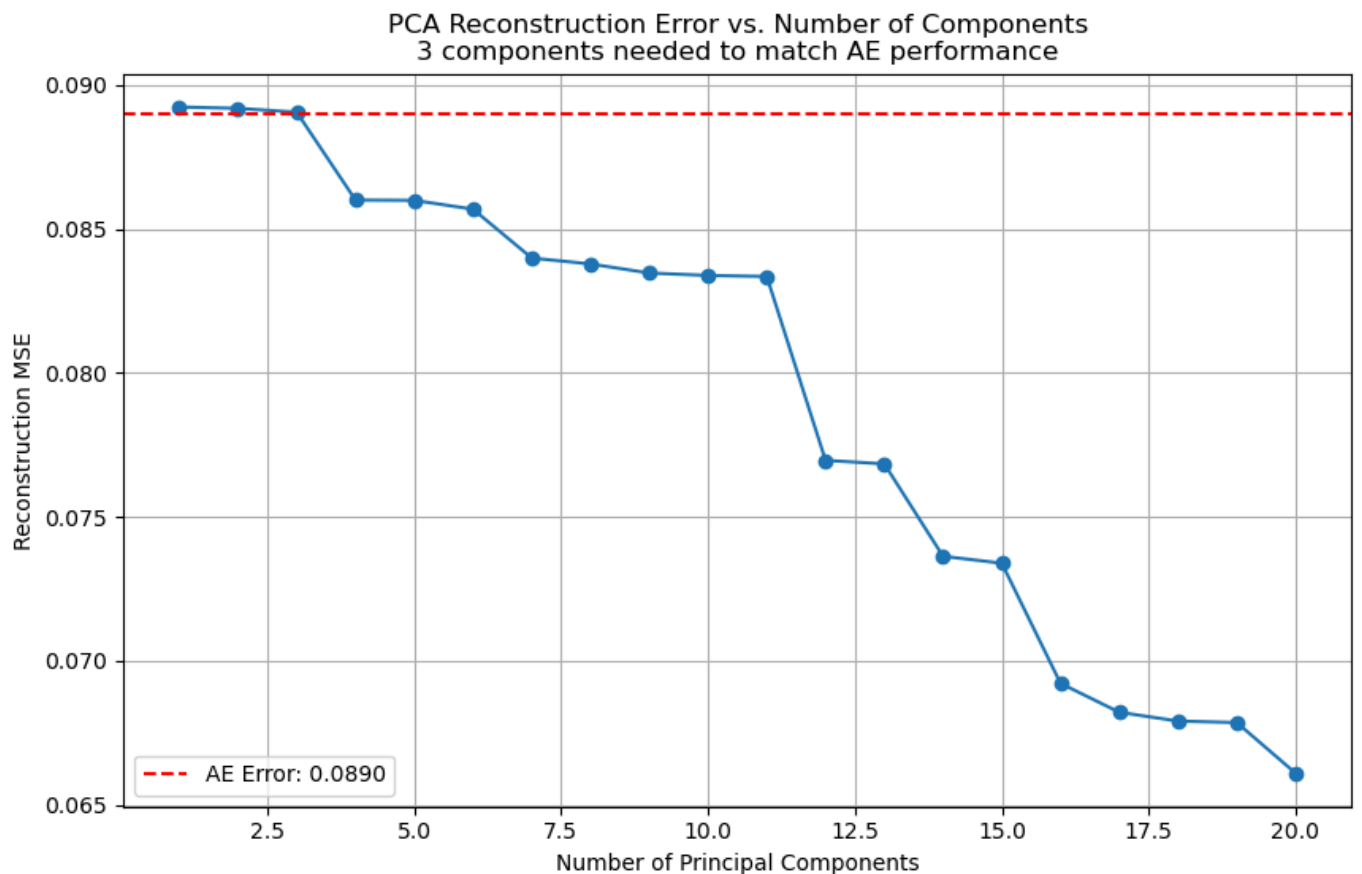
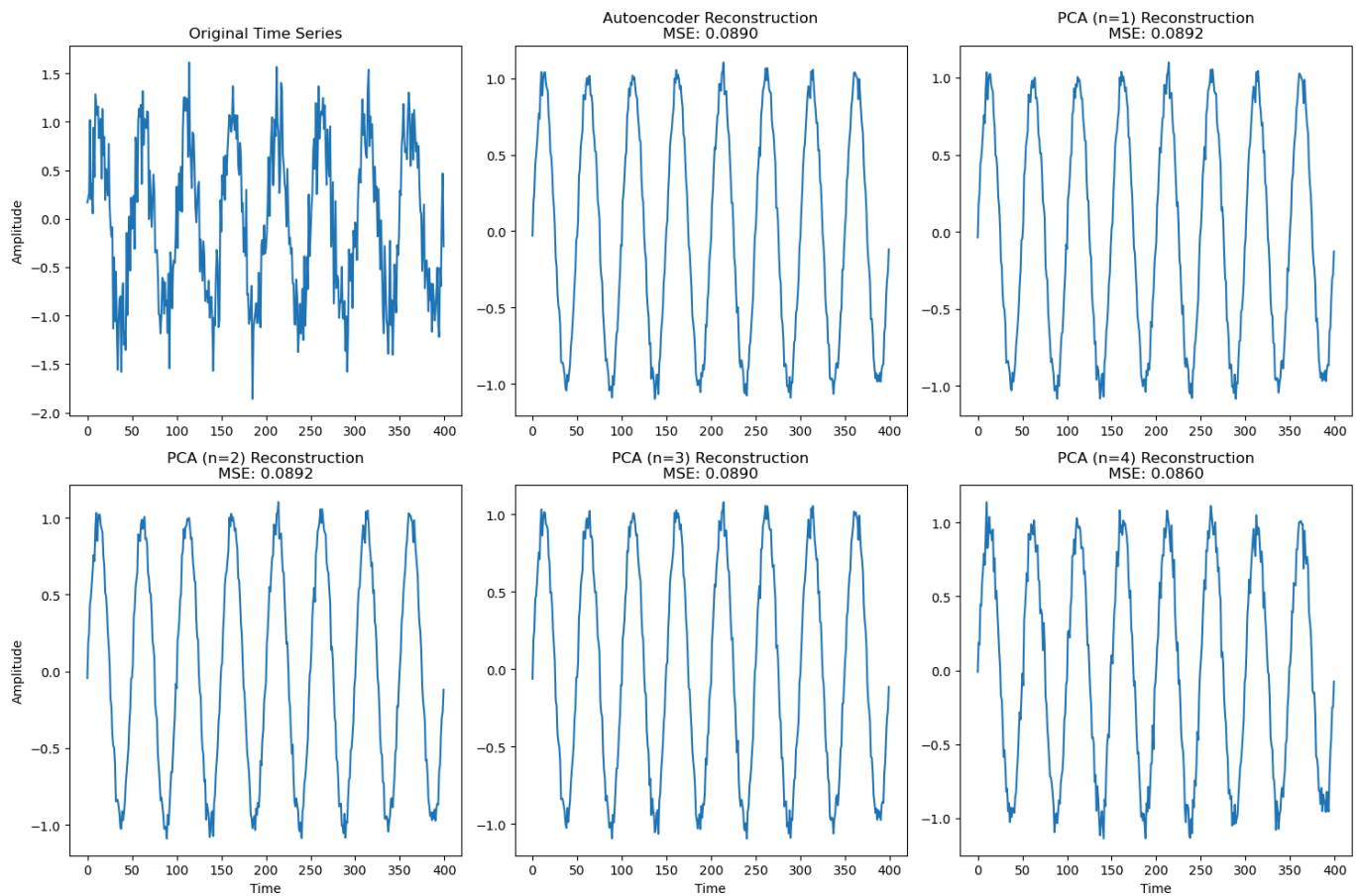
for n in n_components_range:
    # Use the same transformed data, just truncate to n components
    truncated_transformed = np.copy(pca_transformed)
    truncated_transformed[0, n:] = 0
    pca_reconstruction = pca.inverse_transform(truncated_transformed).flatten()
    error = ((single_sample_np - pca_reconstruction)**2).mean()
    all_pca_errors.append(error)

matching_components = np.argmin(np.abs(np.array(all_pca_errors) - ae_error)) + 1

plt.figure(figsize=(10, 6))
plt.plot(n_components_range, all_pca_errors, marker='o')
plt.axhline(y=ae_error, color='r', linestyle='--', label=f'AE Error: {ae_error:.4f}')
plt.xlabel('Number of Principal Components')
plt.ylabel('Reconstruction MSE')
plt.title(f'PCA Reconstruction Error vs. Number of Components\n{n_matching_components}')
plt.grid(True)
plt.legend()
plt.show()

print(f"The autoencoder with {codings.shape[1]} latent dimensions achieves an MSE of")
print(f"To achieve similar performance, PCA needs approximately {matching_components}")

```



The autoencoder with 2 latent dimensions achieves an MSE of 0.0890
 To achieve similar performance, PCA needs approximately 3 principal components

Observation:

The autoencoder with 2 latent dimensions achieves an MSE of 0.0890, while PCA requires approximately 3 principal components to achieve similar reconstruction performance. This suggests that autoencoders can potentially capture more information in fewer dimensions compared to PCA, likely due to their ability to learn non-linear transformations. The non-linear activation functions in

the neural network allow the autoencoder to represent complex patterns more efficiently than PCA's linear transformations.

However, it's important to note that this observation is based on a single sample, which limits the generalizability of our conclusion. A more robust analysis would require evaluating performance across the entire dataset and potentially with different architectures. Additionally, while autoencoders may achieve better compression in this case, PCA offers other advantages such as interpretability and computational efficiency that should be considered when choosing between these dimensionality reduction techniques.